Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat

„Recept Plus”

Készítették:

Bari Dániel,

Molnár László

Ózd, 2026

Tartalomjegyzék

Bevezetés ... 3
Felhasználói dokumentáció ... 4
Rendszerkövetelmény ... 4
Webalkalmazás indítása ... 4
Webalkalmazás használata ... 6
Általános információk a webalkalmazásról ... 6
Reszponzivitás ... 15
Fejlesztői dokumentáció ... 18
Alkalmazott fejlesztői és csoportmunka eszközök ... 18
Adatbázis ... 19
Frontend ... 19
Backend ... 19
Adatbázis ... 20
Modell leírása ... 20
Táblák, kapcsolatok ... 20
E-K diagram ... 25
Frontend ... 26
Komponensek ... 26
Modellek ... 27
Service-ek (fő metódusok) ... 28
Backend ... 33
Modellek ... 33
Controllerek ... 33
Tesztelés ... 47
Backend tesztek ... 47
Frontend tesztek ... 51
Továbbfejlesztési lehetőségek ... 54
Irodalomjegyzék ... 55

Bevezetés

Ez a dokumentáció egy receptmegosztó webalkalmazásról szól, amelyet azért készítettünk, hogy a felhasználók könnyen megoszthassák egymással a kedvenc ételeiket. A rendszer fő része egy egyszerű felület, ahol bárki feltöltheti a saját receptjeit, és mások értékelhetik azokat. Így gyorsan kiderül, mely receptek népszerűek vagy hasznosak.

A webalkalmazás egyik fontos funkciója a tároló modul. Ez arra szolgál, hogy a felhasználók nyomon követhessék, milyen alapanyagok vannak otthon. A rendszer ezek alapján automatikusan ajánl recepteket, amelyeket a meglévő hozzávalókból elkészíthetnek. Ez segít abban, hogy kevesebb étel menjen kárba.

A másik hasznos rész a kalóriaszámoló modul, amely megmutatja, hogy az egyes receptek nagyjából mennyi kalóriát tartalmaznak. Ez jól jön azoknak, akik odafigyelnek az étkezésükre vagy valamilyen diétát követnek.

A jelenlegi receptmegosztó oldalak közül kevésben található meg egyszerre az alapanyagkövetés, a kalóriaszámítás és a közösségi értékelés. A mi rendszerünk ezeket egy helyen kínálja, így egyszerre praktikus és könnyen használható. A cél az volt, hogy a felhasználók gyorsan találjanak nekik megfelelő recepteket, és közben egy aktív közösség részévé válhassanak, így egészségesebben élhetnek.

GitHub elérhetőség: https://github.com/WalkerWalking/ReceptPlus

Felhasználói dokumentáció

Rendszerkövetelmény

Hardver

- PC/laptop: minimum 4 GB RAM, kétmagos processzor
- Mobil eszköz: Android 8.0+ vagy iOS 12.0+

Szoftver

- Visual Studio Code (1.96 vagy újabb)
- Node.js (aktuális verzió)
- XAMPP (8.x verzió)
- Composer (2.x verzió)
- Angular CLI
- Támogatott böngészők:
    - Google Chrome
    - Mozilla Firefox
    - Microsoft Edge
    - Safari

Webalkalmazás indítása

Az alkalmazás futtatásához az alábbi szoftverek telepítése szükséges:

- XAMPP (Apache + MySQL)
- PHP (XAMPP részeként)
- Visual Studio Code
- Composer
- Angular CLI
- Node.js és npm
- Modern webböngésző

XAMPP konfigurálása

1. Indítsuk el a XAMPP Control Panelt
2. Kapcsoljuk be az Apache és MySQL modulokat
3. Nyissuk meg a következő címet: http://localhost/phpmyadmin
4. Ellenőrizzük a működését

Laravel backend indítása

1. Nyissuk meg a projektet Visual Studio Code-ban
2. Ellenőrizzük az .env fájlban az adatbázis beállításokat
3. Nyissunk egy új terminált
4. Futtassuk a következő parancsot: php artisan migrate –seed
5. Futtassuk a következő parancsot: php artisan storage:link
6. Indítsuk el a szervert: php artisan serve
7. A backend ezután elérhető lesz

Angular frontend indítása

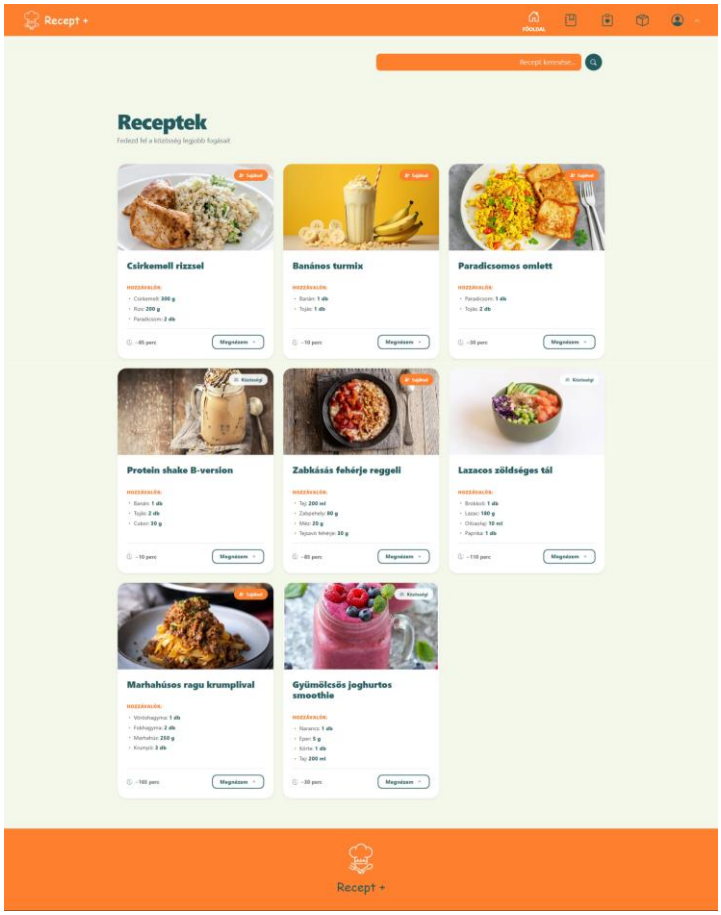
1. Nyissuk meg a frontend mappát Visual Studio Code-ban
2. Nyissunk egy új terminált
3. Telepítsük a függőségeket: npm install
4. Indítsuk el a fejlesztői szervert: ng serve
5. A frontend elérhető lesz a böngészőben (általában http://localhost:4200)

Webalkalmazás használata

Általános információk a webalkalmazásról

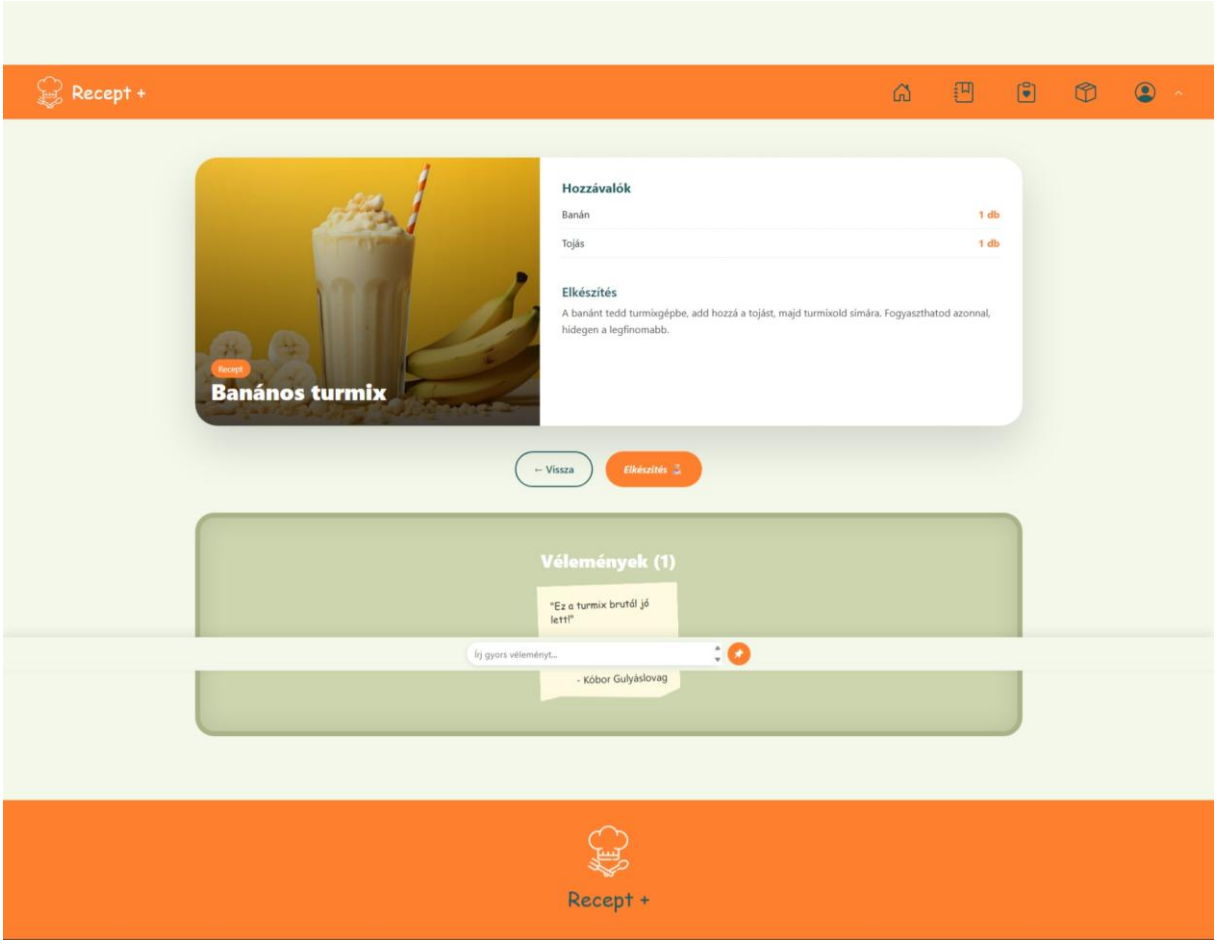
Az alkalmazás teljes funkcióinak eléréséhez regisztráció, vagy bejelentkezés szükséges, ha már van fiókunk. Az alkalmazás bejelentkezés nélkül is használható, ám csak receptek megtekintésére. Bejelentkezés után a webalkalmazás képes követni a bevitt kalóriánkat, kiszámolja a testtömeg-indexünket, a napi ajánlott kalóriabevitelt, és képes számontartani az otthon tárolt hozzávalóinkat. A receptek alatt kommentek is találhatóak. Egy bejelentkezett felhasználó hozzá tud adni recepteket az adatbázishoz, valamint a saját adatait is módosítani tudja. A recepteknél található egy „Elkészítettem” gomb, ami levonja a hozzávalóinkból a recept hozzávalóit, valamint hozzáadja a megevett kalóriánkhoz a recept kalóriaértékét.

Főoldal



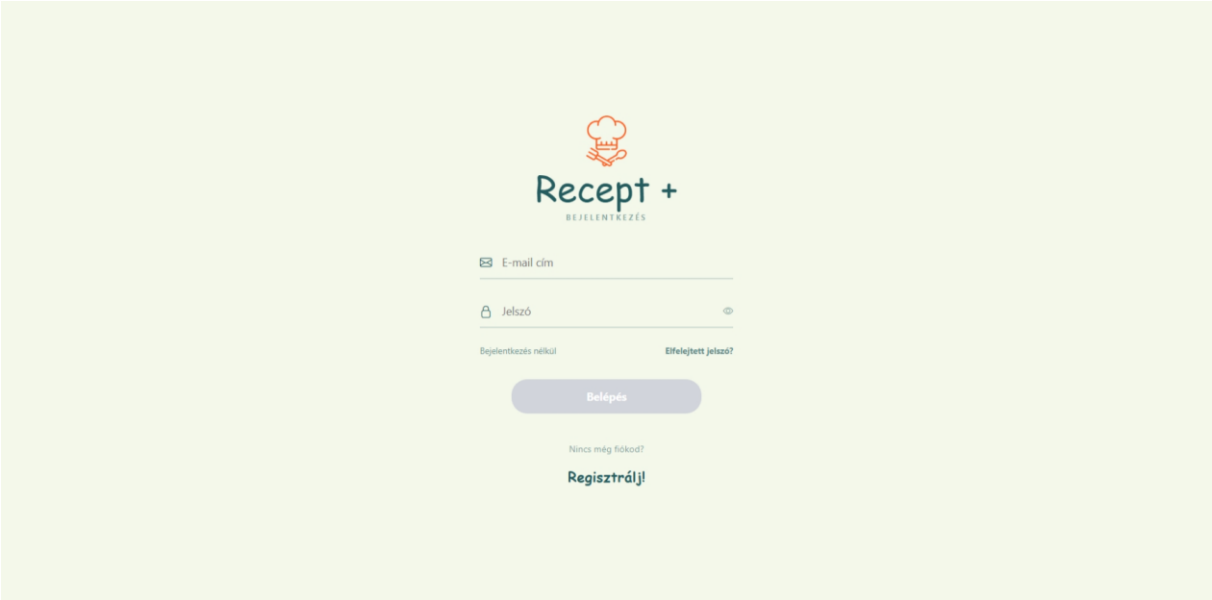
Kép 1 - Főoldal képernyőképe

A főoldal bejelentkezés nélkül is elérhető. Itt található az összes recept, egy kártyára kattintva pedig megjelenik a teljes recept kommentekkel együtt. Itt van lehetőség komment írására (a kommentek névtelenek), és a recept elkészítésére, ha bejelentkezett a felhasználó.



Kép 2 - Recept felület képernyőképe

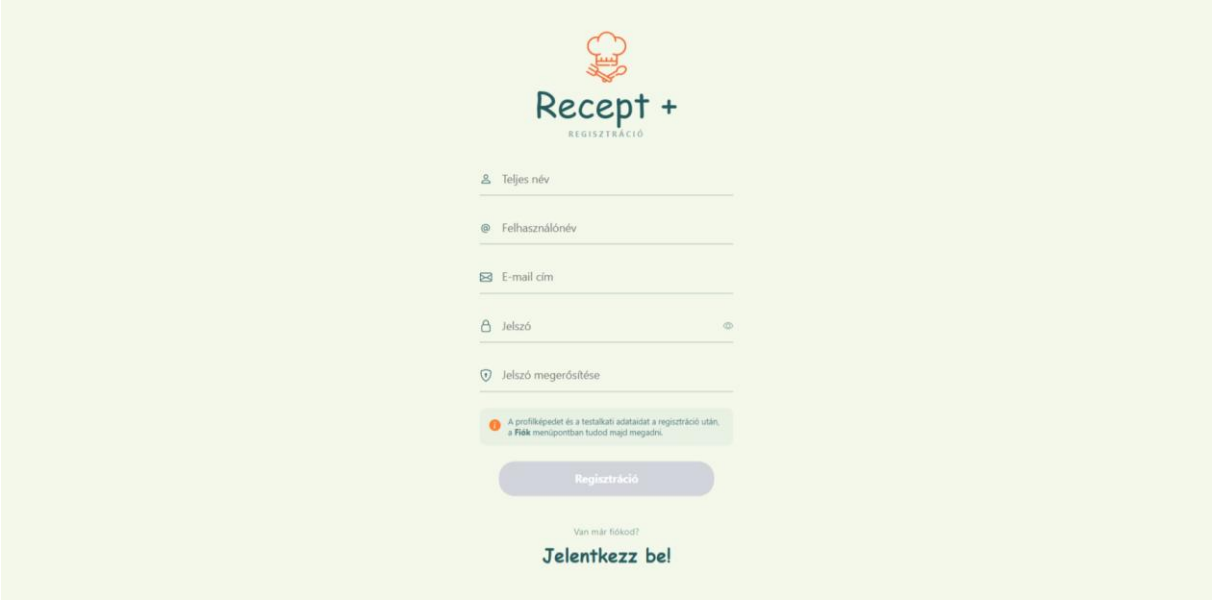
Bejelentkezés oldal



Kép 3 - Bejelentkezés oldal képernyőképe

A bejelentkezés oldal egy letisztult felület, ami egy adott felhasználó bejelentkeztetésére szolgál. Elfelejtett jelszó esetén email-t küld az alkalmazás, hogy ilyen esetekben is be tudjunk jelentkezni.

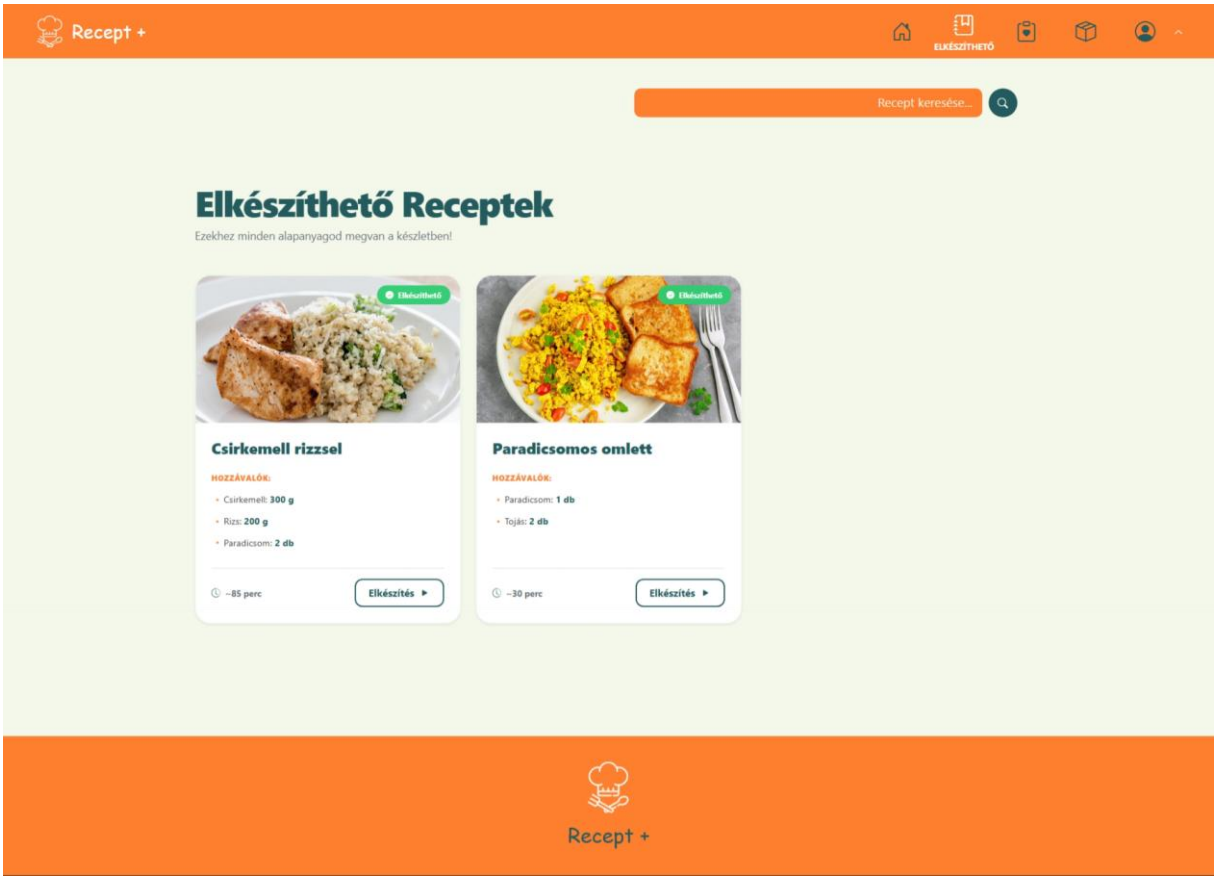
Regisztráció oldal



Kép 4 - Regisztráció oldal képernyőképe

A felület egy felhasználó regisztrálását teszi lehetővé. Itt nem minden adat kerül megadásra, néhány adatot csak a későbbiekben a „Fiók” oldalon tudunk beállítani.

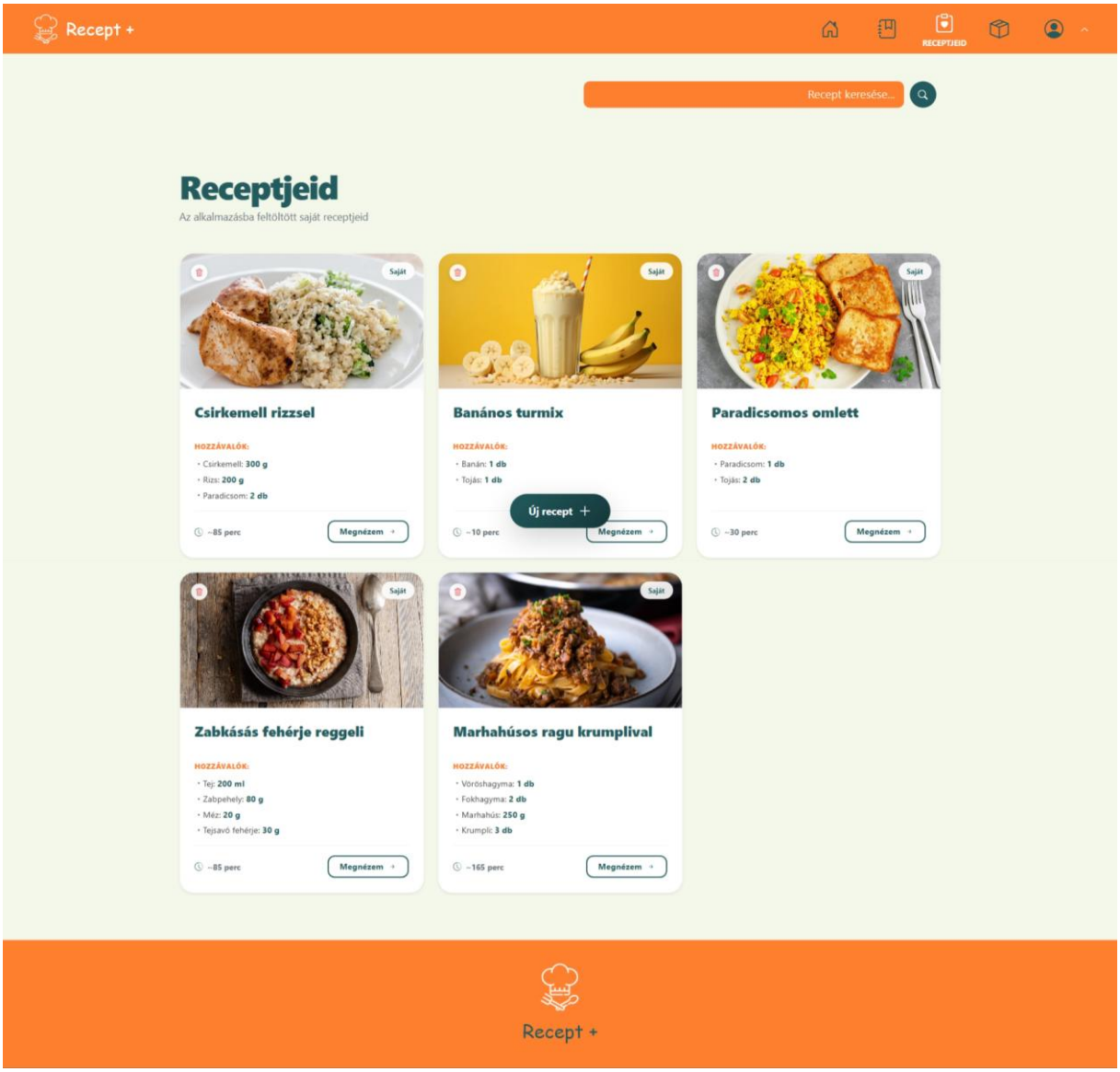
Elkészíthető receptek oldal



Kép 5 - Elkészíthető receptek oldal képernyőképe

Lényegében ugyanaz, mint a főoldal, csak itt a saját alapanyagainkból elkészíthető recepteket látjuk. A kártyára kattintva ugyanarra a felületre visz, mint a főoldalról.

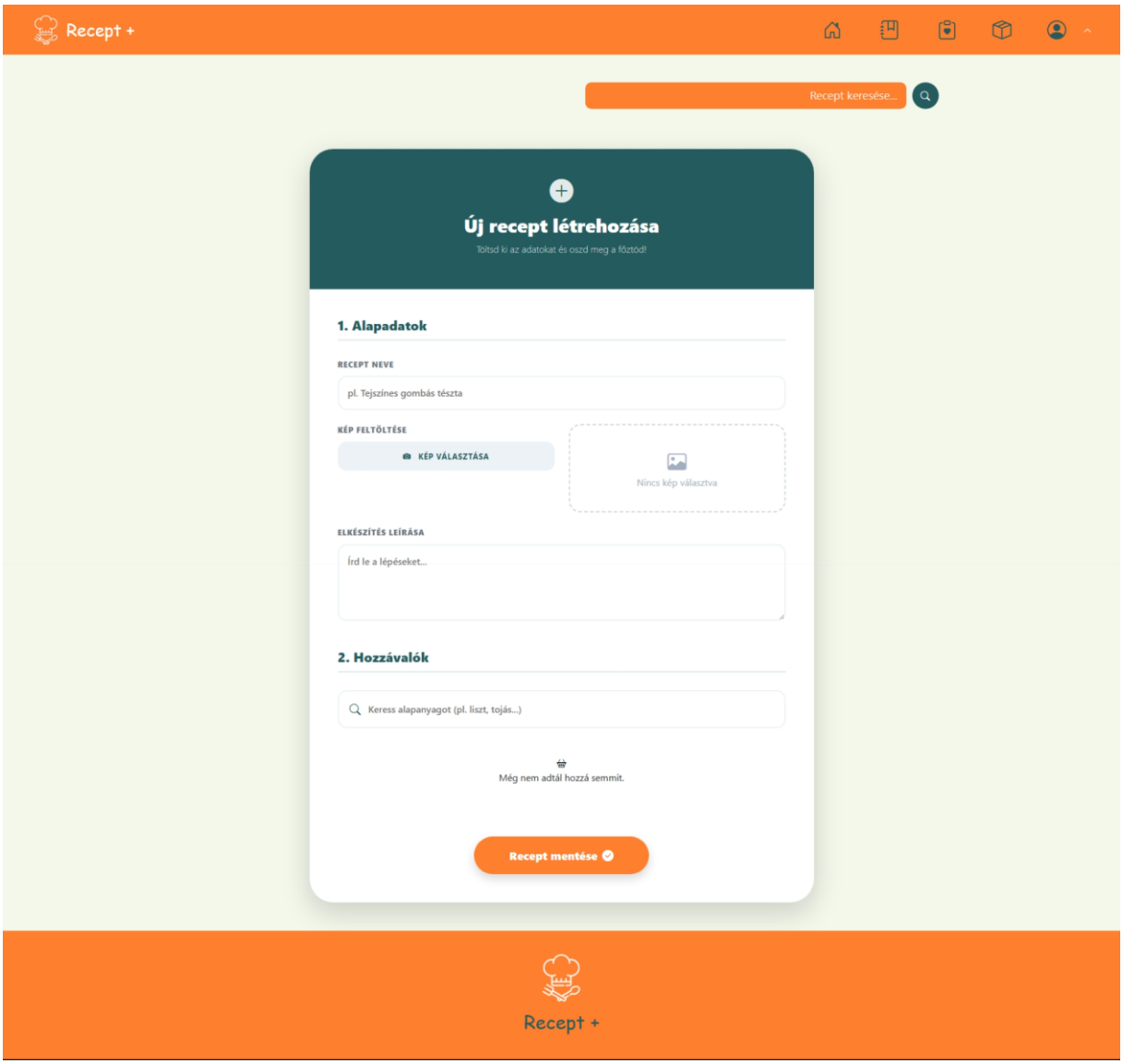
Saját receptek oldal



Kép 6 - Saját receptek oldal képernyőképe

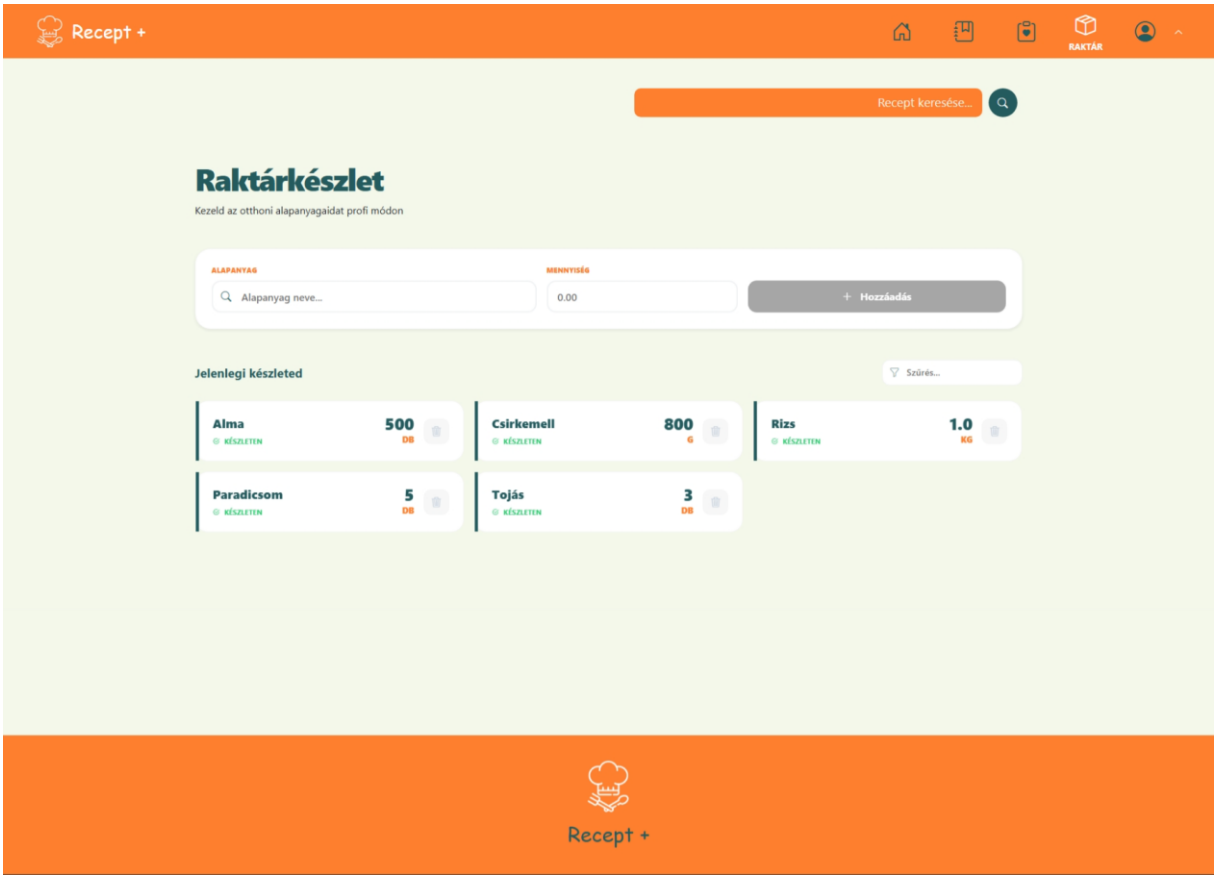
Hasonló a főoldal és az elkészíthető receptek felületekhez, de itt az adott felhasználó által feltöltött recepteket látjuk.

Található az oldalon egy gomb, ami egy felületre viszi a felhasználót, ahol hozzá tud adni recepteket a webalkalmazáshoz.



Kép 7 - Recept hozzáadás oldal képernyőképe

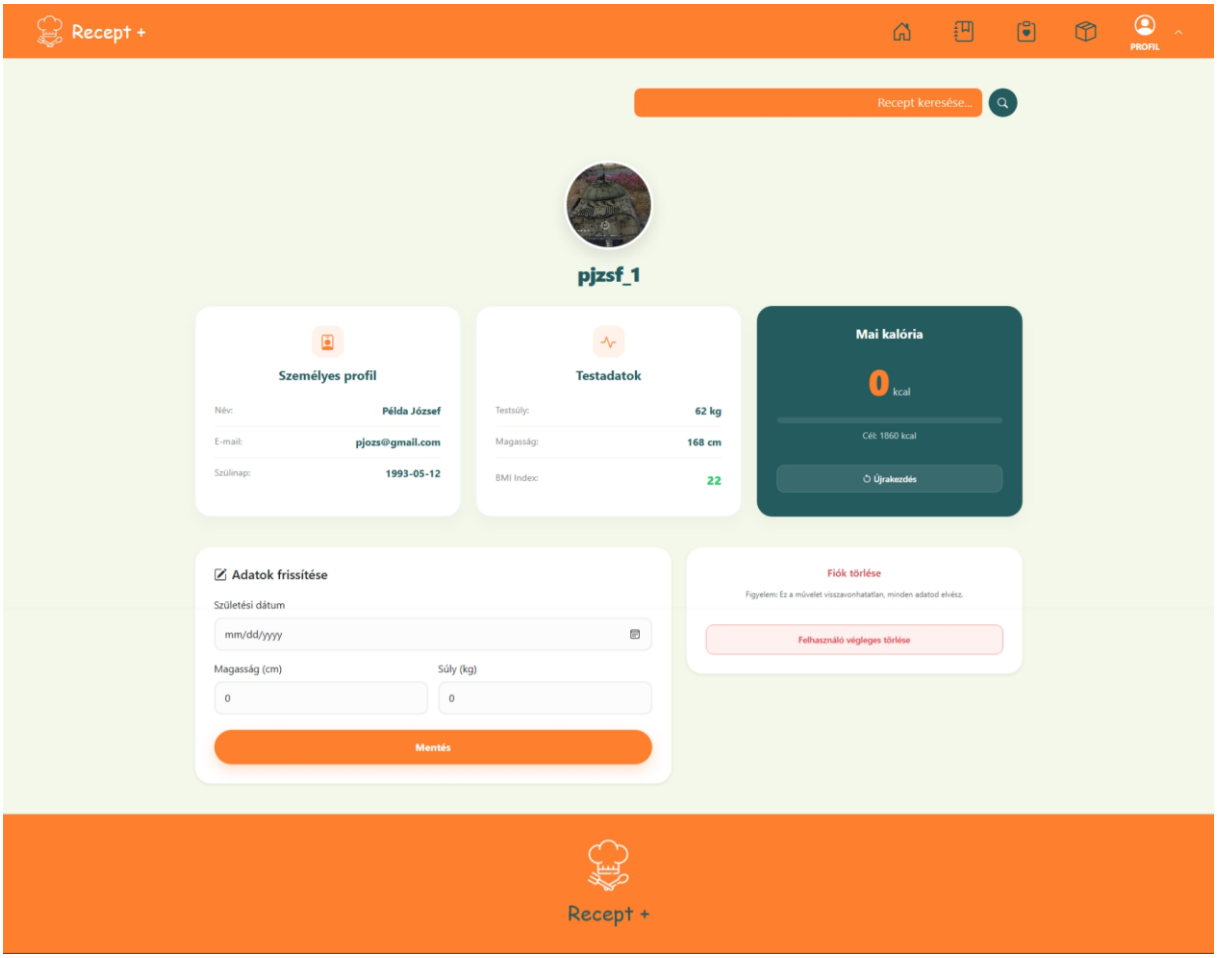
Hozzávaló tároló oldal



Kép 8 - Tároló oldal képernyőképe

Ezen a felületen található a felhasználó összes hozzávalója, itt tud hozzáadni, módosítani, törölni hozzávalókat. Csak előre megadott alapanyagokból lehet válogatni, a korlátozás az alkalmazás moderálására ad lehetőséget, elkerülve a helytelen hozzávalókat. A mennyiség csak nullától nagyobb lehet, átírásra csak törléssel és újraírással van lehetőség.

Fiók oldal



Kép 9 - Fiók oldal képernyőképe

Ezen az oldalon kerülnek a felhasználó adatai megjelenítésre, valamint további adatokat tud módosítani a felhasználó, így az alkalmazás ki tudja számítani a felhasználó testtömeg-indexét, és ajánlott napi kalóriabeviteli mennyiségét. A fiókunkat itt lehet törölni is.

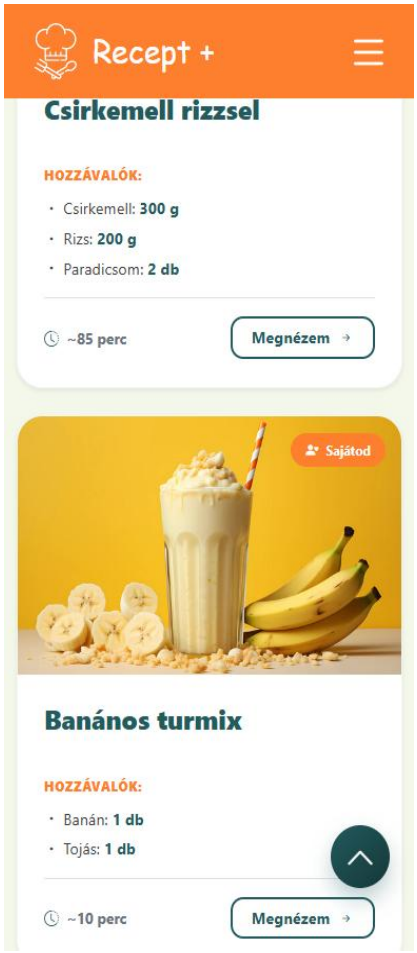
Rólunk oldal

Kép 10 - Rólunk oldal képernyőképe

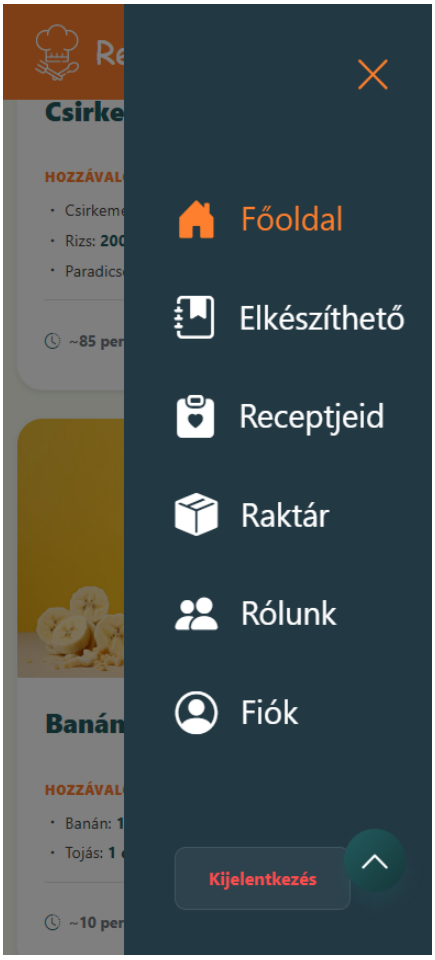
Ezen az oldalon található rólunk, az alkalmazás fejlesztőiről egy-egy rövid leírás.

Reszponzivitás

Az oldal mobilnézetben is megfelelő kezelhetőséget biztosít a felhasználók számára. A menü dinamikusan igazodik az adott eszköz kijelzőjének a méretéhez, a különböző kártyák és felületek pedig egymás alá helyezkedve teljesen kitöltik az oldalt (margóval).



Kép 11- Menü mobilnézet képernyőkép

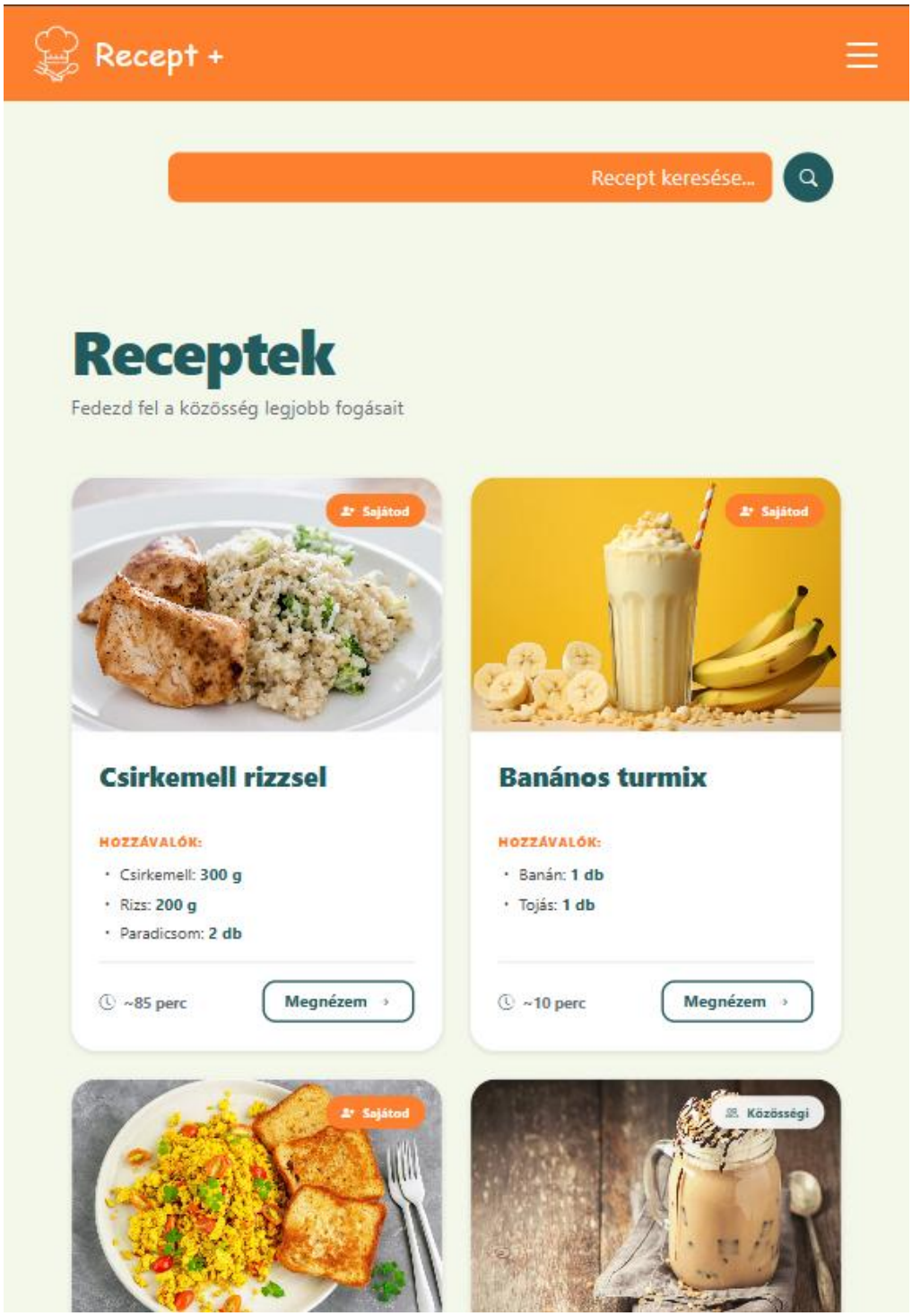


Kép 12 - Főoldal mobilnézet képernyőkép

Kép 13 - Fiók oldal mobilnézet képernyőkép

Kép 14 - Raktár oldal mobilnézet képernyőkép

A tablet nézet is hasonlóan épül fel, mint a mobilnézet:



Kép 15 - Tablet nézet példa

Fejlesztői dokumentáció

Alkalmazott fejlesztői és csoportmunka eszközök

A projekt fejlesztése során több különböző eszközt alkalmaztunk annak érdekében, hogy a munkafolyamat hatékony és jól szervezett legyen, valamint a csapaton belüli kommunikáció is gördülékenyen működjön.

- Git/GitHub

  A forráskód tárolására és verziókezelésére használtuk. Segítségével több fejlesztő is tudott egyszerre dolgozni a projekten, miközben a módosítások visszakövethetők maradtak.
- Discord

  A csapaton belüli kommunikáció fő eszköze volt. Lehetővé tette a gyors üzenetváltást, valamint hang- és videóhívások lebonyolítását.
- Trello

  A feladatok rendszerezésére és nyomon követésére használtuk. A feladatokat kártyák formájában kezeltük, így mindenki láthatta az aktuális állapotot és a saját teendőit.
- VisualStudioCode

  A fejlesztés fő eszköze volt, amely több programozási nyelvet és technológiát is támogat.
- Figma

  A felhasználói felület megtervezésére szolgált. Segítségével előre láthatóvá vált az alkalmazás kinézete.

Adatbázis

- Visual Studio Code

  Az SQL lekérdezések és adatbázis struktúrák megírására használt fejlesztői környezet.
- XAMPP

  A lokális szerver környezet biztosítására szolgált, amely tartalmazza az Apache webszervert és a MySQL adatbázist is.
- MySQL

  Az alkalmazás adatainak tárolására használt adatbázis-kezelő rendszer. Itt kerülnek tárolásra a felhasználók, receptek, hozzávalók és ezek kapcsolatai.

Frontend

- Visual Studio Code

  A kliensoldali fejlesztés fő eszköze.
- Angular

  A frontend keretrendszer, amely lehetővé teszi dinamikus és modern felhasználói felület kialakítását.
- Figma

  A webalkalmazás kinézetének előzetes megtervezésére használt eszköz.

Backend

- Visual Studio Code

  A backend kód és API fejlesztésére használtuk.
- XAMPP

  A lokális adatbázis és szerver futtatására.
- Laravel

  A backend logika és az API megvalósítására használt PHP keretrendszer.

Adatbázis

Modell leírása

A rendszer működését egy több táblából álló relációs adatbázis biztosítja. Az adatbázis fő célja, hogy tárolja a felhasználók adatait, a recepteket, az alapanyagokat, illetve az ezekhez kapcsolódó információkat, mint például a kommenteket és a felhasználó "tárhelyét".

Táblák, kapcsolatok

Users tábla

A Users tábla a rendszerben regisztrált felhasználók alapadatait tárolja. Ebben találhatók a bejelentkezéshez szükséges adatok, például a felhasználónév, az e-mail cím és a jelszó, valamint a személyes profilhoz kapcsolódó információk is, mint a név, testsúly, magasság, születési dátum és profilkép. A tábla szerepe, hogy minden felhasználóhoz egyedi azonosítót biztosítson, amelyre a rendszer többi táblája is hivatkozhat.

Mező	Típus	Leírás
id	int	A felhasználó egyedi azonosítója
passwrd	varchar(150)	Jelszó (titkosított formában)
email	varchar(100)	Felhasználó e-mail címe
name	varchar(100)	Felhasználó teljes neve
username	varchar(50)	Bejelentkezési felhasználónév
bodyweightKg	int	Testsúly (kg)
heightCm	int	Magasság (cm)
birthDate	int	Születési év
profilePictureURL	varchar(200)	Profilkép URL-je
caloriesEaten	int	Megevett kalória mennyisége

Tábla 1 - Users adatbázis tábla

Ingredients tábla

Az Ingredients tábla az alkalmazásban használható hozzávalókat tartalmazza. Minden hozzávalóhoz eltárolásra kerül annak neve, típusa (például gyümölcs-e), kalóriaértéke és mértékegysége. Ez a tábla biztosítja, hogy a receptek és a felhasználói készletek egységes hozzávalólistára épüljenek, így elkerülhető a duplikáció és az eltérő megnevezések használata.

Mező	Típus	Leírás
id	int	A hozzávaló egyedi azonosítója
name	varchar(100)	A hozzávaló neve
isFruit	boolean	Egy logikai típus, amely azt dönti el, gyümölcs-e a hozzávaló
kcalPerGram	float	A kalória mennyisége 1 gramban
unit	varchar(255)	A hozzávaló mértékegysége

Tábla 2 - Ingredients adatbázis tábla

Recipes tábla

A Recipes tábla a felhasználók által létrehozott recepteket tárolja. Minden recepthez tartozik név, létrehozó felhasználó, leírás, kép, kalóriaérték, valamint egy logikai mező, amely jelzi, hogy az adott recept elkészíthető-e a rendelkezésre álló alapanyagokból. Ez a tábla az alkalmazás egyik központi eleme, hiszen a receptek köré épül a hozzávalók, kommentek és felhasználói interakciók kezelése.

Mező	Típus	Leírás
id	int	A recept egyedi azonosítója
name	varchar(200)	A recept neve
userId	int	Idegen kulcs. Arra a felhasználóra mutat, aki létrehozta a receptet az oldalon

Tábla 3 - Recipes adatbázis tábla

Comments tábla

A Comments tábla a receptekhez írt hozzászólásokat kezeli. Ebben a táblában tárolódik a komment szövege, valamint az, hogy melyik recepthez és melyik felhasználóhoz tartozik az adott hozzászólás. Segítségével a felhasználók visszajelzést adhatnak a receptekről, így közösségi funkcióval bővíti az alkalmazást.

Mező	Típus	Leírás
id	int	A recept egyedi azonosítója
content	varchar(3500)	A komment tartalma
recipeId	int	Idegen kulcs. A receptre hivatkozik, amelyhez az adott komment került.
userId	int	Idegen kulcs. Arra a felhasználóra mutat, aki létrehozta a kommentet az adott receptnél

Tábla 4 - Comments adatbázis tábla

User_ingredient tábla

A User_ingredient tábla a felhasználók saját hozzávaló-készletét, vagyis a “tárhely” részt valósítja meg. A tábla azt tárolja, hogy egy adott felhasználó milyen hozzávalóval rendelkezik, és abból mekkora mennyiség áll a rendelkezésére. Ez a kapcsolat lehetővé teszi, hogy az alkalmazás meg tudja állapítani, mely receptek készíthetők el a felhasználó jelenlegi alapanyagaiból.

Mező	Típus	Leírás
userId	int	Idgen kulcs. A felhasználóra mutat.
ingredientId	int	Idegen kulcs. A hozzávalóra mutat
amount	int	A felhasználó rendelkezésre álló mennyisége az adott hozzávalóból

Tábla 5 - User_ingredient adatbázis tábla

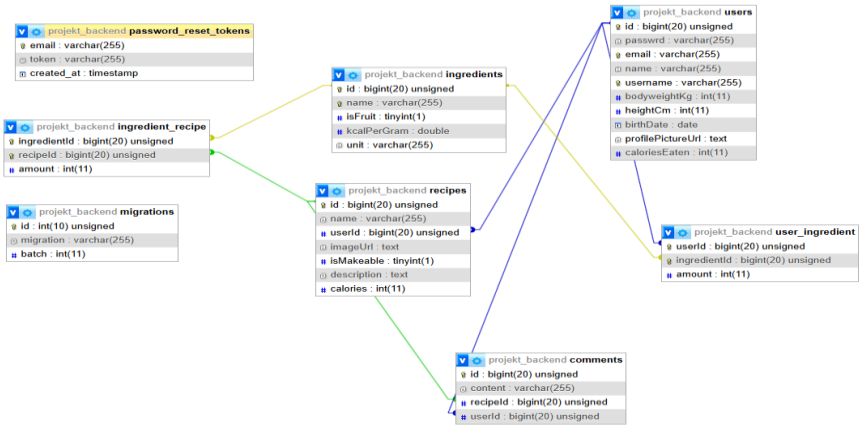
Ingredient_recipe tábla

Az Ingredient_recipe tábla a receptek és a hozzávalók közötti kapcsolatot kezeli. Azt rögzíti, hogy egy adott recepthez mely hozzávalók tartoznak, illetve azokból milyen mennyiség szükséges az elkészítéshez. Ez a tábla teszi lehetővé a recept pontos összetevőlistájának kezelését, valamint az elkészíthetőség és kalóriaszámítás funkciók működését.

Mező	Típus	Leírás
userId	int	Idgen kulcs. A felhasználóra mutat
recipeId	int	Idegen kulcs. A receptre mutat
gramAmount	int	A recepthez szükséges mennyiség az adott hozzávalóból

Tábla 6 - Ingredient_recipe adatbázis tábla

Kapcsolatok:



Kép 16 - Adatbázis tábla kapcsolatok

A users és recipes tábla között 1-sok kapcsolat található, a userId idegen kulcs mutat a users tábla id elsődleges kulcsára. Egy felhasználóhoz több recept tartozhat, egy receptet két felhasználó nem tud feltölteni.

A comments és recipes táblák között szintén 1-sok kapcsolat áll fenn: a recipeId idegen kulcs a recipes.id mezőre mutat. Egy recepthez több hozzászólás tartozhat, de egy hozzászólás csak egyetlen recepthez köthető.

A comments tábla a users táblára is hivatkozik: a userId mező idegen kulcs a users.id-re. Ez azt jelenti, hogy egy felhasználó több kommentet is írhat, de egy kommentet csak egyetlen felhasználó írhat. Így a hozzászólások mindig egy adott felhasználóhoz és egy adott recepthez kapcsolódnak.

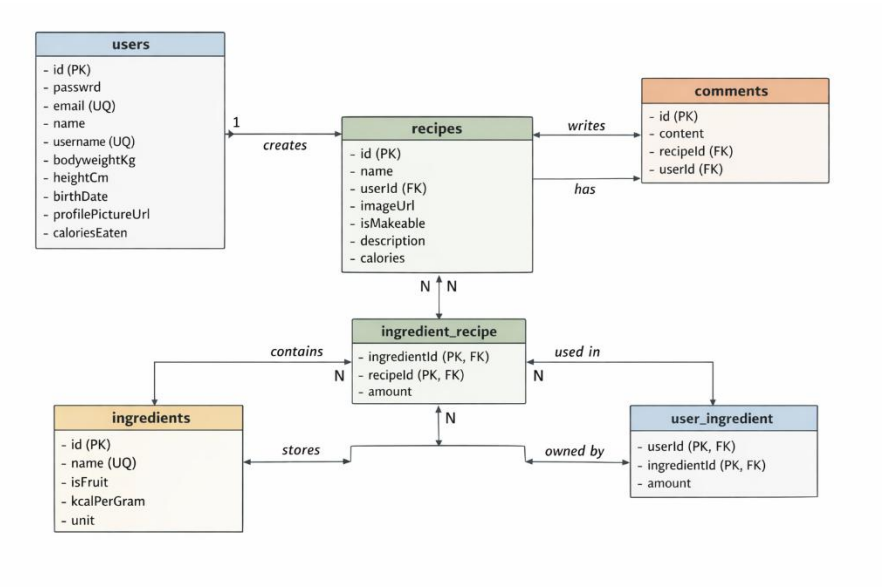
A recipes és ingredients táblák között több-több kapcsolat található, amit az ingredient_recipe kapcsolótábla valósít meg. Ebben a táblában az ingredientId és recipeId együtt alkotják az összetett elsődleges kulcsot. Egy recept több hozzávalót is tartalmazhat, és egy hozzávaló több receptben is felhasználható. A gramAmount mező rögzíti, hogy egy adott hozzávalóból mennyi szerepel az adott receptben.

A users és ingredients táblák között szintén több-több kapcsolat van, amelyet a user_ingredient tábla kezel. Ennek célja a felhasználók saját hozzávaló-készletének (pl. otthoni alapanyagok) nyilvántartása. A táblában a userId és ingredientId alkotják az összetett elsődleges kulcsot, a gramAmount pedig az adott hozzávaló mennyiségét tárolja. Egy

felhasználónak több hozzávalója is lehet, és természetesen ugyanaz a hozzávaló több felhasználónál is előfordulhat.

A ingredients tábla az összes alapanyagot tartalmazza, olyan attribútumokkal, mint az isFruit (boolean jellegű, gyümölcs-e az adott hozzávaló) és a kcalPerGram (egy gramra jutó energiatartalom). A táblát több más tábla is idegen kulccsal hivatkozza, de az összes kapcsolat több-több jellegű, így közvetlenül nem tartalmaz recept- vagy felhasználó hivatkozást (idegen kulcsot).

E-K diagram



Kép 17 - E-K diagram

Frontend

Az alkalmazás frontend része az Angular keretrendszerrel készült, amely egy komponens alapú keretrendszer. A frontend feladata a felhasználói felület megjelenítése, valamint a backend API-val való kommunikáció HTTP kéréseken keresztül.

Komponensek

A komponensek főként a service-ek metódusaival dolgoznak, külön saját komponensbeli metódusok csak azok meghívására szolgálnak.

aboutuscomponent: A rólunk oldal komponense, itt csak html szöveg található.

accountinfo: A felhasználó fiókjának beállítása, valamint a kalóriaszámláló ezen a felületen található.

add-recipe: Egy recept feltöltésére szolgáló felület, komponens.

all-recipes: Gyakorlatilag a főoldal, itt található az alkalmazás összes receptje. Ezt látja a felhasználó bejelentkezés nélkül is.

footer: Az alkalmazás footere, csak egy logót tartalmaz.

login: A bejelentkezési felület komponense.

navbar: A navigációs menü megjelenítésére szolgáló komponens.

recepiesfrominventory: A felhasználó elkészíthető receptjeinek megjelenítésére szolgáló komponens.

recipe-full: Egy receptre való kattintáskor ez a komponens töltődik be, egy teljes receptet jelenít meg kommentekkel együtt.

register: A regisztrációra szolgáló komponens, innen tölti fel a felhasználó az adatait, amik szükségesek a fiók létrehozásához.

resetpasswordcomponent: A felhasználó jelszavának visszaállítására szolgáló komponens.

searchbar: A komponens, amely tartalmazza a keresősávot, ez a navbar és a footer komponensekkel együtt állandó megjelenítéssel az oldalon helyezkedik el.

toastwindowcomponent: A felugró kártyák megjelenítésére szolgáló komponens.

userrecepies: A felhasználó által létrehozott receptek megjelenítésére szolgáló komponens, itt lehet törölni a különböző recepteket, melyek a bejelentkezett fiókhoz kapcsolódnak.

userstorage: A felhasználó által eltárolt alapanyagok megjelenítésére szolgáló komponens, itt lehet hozzáadni és törölni saját alapanyagokat.

Modellek

Comment modell: Rendelkezik id, content, receipeId, userId mezőkkel, így könnyen kezelhetőek a feltöltött kommentek.

Ingredient modell: Rendelkezik id, name, isFruit, kcalPerGram, unit mezőkkel. Ezek a mezők a dinamikus megjelenítést támogatják, valamint lehetővé teszik a kalóriát és elfogyasztott hozzávalókat használó számolások működését.

Recipe modell: Rendelkezik id, name, userId, imageUrl, isMakeable, description, calories és ingredients (modell) mezőkkel. Az ingredients modell (recept-hozzávaló) rendelkezik id, name, amount, unit mezőkkel. Ezek a mezők a receptek és hozzávalóik megjelenítését szolgálják, valamint az elfogyasztott hozzávalók számolásához elengedhetetlen adatokat tartalmaznak.

User modell: rendelkezik id, passwrd, email, name, username, bodyweightKg, heightCm, birthDate, profilePictureUrl, caloriesEaten, bmi (opcionális), calorieGoal (opcionális) mezőkkel. Ezek a mezők a felhasználó bejelentkeztetését teszik lehetővé, valamint a bmi és a napi bevitt kalória, illetve kalória - cél megjelenítését szolgálják. A további mezők a felhasználó egyéb adatait kezelik (pl.: profilkép elérési útja).

UserIngredient modell: rendelkezik id, name, amount, unit mezőkkel. A felhasználó által tárolt alapanyagok kezelésére szolgáló modell.

Service-ek (fő metódusok)

Auth Service

Az AuthService feladata, hogy a frontend és a Laravel backend API között kapcsolatot teremtsen a bejelentkezés, regisztráció, jelszó-visszaállítás, felhasználói adatok kezelése és a kijelentkezés során. A szolgáltatás a HttpClient segítségével küld HTTP kéréseket a backend felé, míg az alkalmazás aktuális bejelentkezési állapotát Angular signal alapú változókban tárolja.

getUser: A getUser() metódus a felhasználó bejelentkezését valósítja meg. A megadott e-mail címet és jelszót elküldi a backend /login végpontjára, majd sikeres válasz esetén eltárolja a bejelentkezett felhasználó adatait és frissíti az alkalmazás bejelentkezési állapotát. A metódus a felhasználói adatokat localStorage-ban is menti, valamint értesítést jelenít meg a sikeres belépésről. Hiba esetén a különböző státuszkódok alapján megfelelő hibaüzenetet jelenít meg.

```
getUser(userEmail: string, userPassword: string): Observable<User> {
  return this.http.post<any>(`${this.API_URL}/login`, { email: userEmail, passwrd: userPassword }).pipe(
    tap((user) => {
      this.loggedInUser.set(user);
      this.isLoggedIn.set(true);
      localStorage.setItem('loggedInUser', JSON.stringify(user));
      this.toast.show(`Üdv újra, ${user.username}!`, 'success');
    }),
    catchError((err: HttpErrorResponse) => {
      let errorMessage = 'Hiba a bejelentkezésnél!';
      if (err.status === 401) errorMessage = 'Hibás email cím vagy jelszó!';
      else if (err.status === 404) errorMessage = 'Nincs ilyen felhasználó!';
      else if (err.status === 0) errorMessage = 'A szerver nem elérhető!';

      this.toast.show(errorMessage, 'error');
      return throwError(() => err);
    })
  );
}
```

Kép 18- getUser frontend metódus

registerUser: A registerUser() metódus új felhasználó létrehozását végzi el. A megadott felhasználói adatokat elküldi a backend /register végpontjára, majd sikeres regisztráció esetén visszajelzést ad a felhasználónak. Hiba esetén kezeli a szerver válaszát, és megfelelő hibaüzenetet jelenít meg, például akkor, ha az e-mail cím vagy a felhasználónév már foglalt.

```
registerUser(user: { email: string; passwrd: string; name: string; username: string; }): Observable<any> {
  return this.http.post<any>(`${this.API_URL}/register`, user).pipe(
    tap(() => this.toast.show('Sikeres regisztráció!', 'success')),
    catchError((err: HttpErrorResponse) => {
      let errorMessage = 'A regisztráció nem sikerült.';
      if (err.status === 422) errorMessage = 'Ez az email vagy felhasználónév már foglalt!';
      this.toast.show(errorMessage, 'error');
      return throwError(() => err);
    })
  );
}
```

Kép 19 - registerUser frontend metódus

Comment Service

A CommentService osztály az Angular alkalmazásban a hozzászólások kezeléséért felelős szolgáltatás. Feladata, hogy kapcsolatot biztosítson a frontend és a Laravel backend API között a kommentek lekérése, létrehozása és törlése során. A szolgáltatás a HttpClient segítségével kommunikál a szerverrel, valamint felhasználói visszajelzéseket jelenít meg különböző műveletek eredményéről.

getCommentsByRecipeId: A getCommentsByRecipeId() metódus egy adott recepthez tartozó hozzászólásokat kéri le a backendből. A megadott receptazonosító alapján HTTP GET kérést küld, majd visszaadja a recepthez tartozó kommentek listáját. Ha az adott recepthez nem található hozzászólás, a metódus üres tömböt ad vissza. Egyéb hiba esetén értesítést jelenít meg a felhasználónak.

```
getCommentsByRecipeId(recipeId: number): Observable<Comment[]> {
  return this.http.get<Comment[]>(`http://127.0.0.1:8000/api/comments/getByRecipeId/${recipeId}`).pipe(
    catchError((error) => {
      if (error.status === 404) {
        return of([]);
      }
      this.toast.show('Nem sikerült betölteni a hozzászólásokat.', 'error');
      return throwError(() => error);
    })
  );
}
```

Kép 20 - getCommentsByRecipeId frontend metódus

createComment: A createComment() metódus új hozzászólás létrehozását végzi el. A megadott kommentadatokat HTTP POST kérés formájában elküldi a backendnek, majd sikeres mentés esetén értesítést jelenít meg a felhasználónak. Hiba esetén a rendszer hibaüzenetet ad vissza.

```
createComment(comment: { content: string; recipeId: number; userId: number }): Observable<any> {
  return this.http.post<any>('http://127.0.0.1:8000/api/comments', comment).pipe(
    tap(() => {
      this.toast.show('Hozzászólás elküldve!', 'success');
    }),
    catchError((err) => {
      this.toast.show('Hiba történt a küldés során.', 'error');
      return throwError(() => err);
    })
  );
}
```

Kép 21 - createComment frontend metódus

Recipe Service

A RecipeService osztály az Angular alkalmazás egyik központi szolgáltatása, amely a receptek kezeléséért, szűréséért és feldolgozásáért felelős. A szolgáltatás biztosítja a kapcsolatot a frontend és a Laravel backend API között, valamint kezeli a receptek állapotát, a keresést, illetve az elkészíthetőségi logikát. A működés során Angular signal és computed alapú reaktív állapotkezelést használ, ami lehetővé teszi az adatok automatikus frissítését a felhasználói felületen.

makeableRecipes: A makeableRecipes egy számított lista, amely azokat a recepteket tartalmazza, amelyek a felhasználó aktuális hozzávalói alapján elkészíthetők. A lista szükség esetén keresési feltétel alapján is szűrhető, így a felhasználó receptnév vagy hozzávaló szerint is kereshet az elkészíthető receptek között.

```
makeableRecipes = computed(() => {
  const recipes = this.recipes();
  const userIngredients = this.storage.userIngredients();
  const term = this.searchTerm().toLowerCase().trim();

  if (!recipes.length || !userIngredients.length) return [];
  const canMake = recipes.filter(recipe => this.isMakeable(recipe, userIngredients));

  if (term.length < 3) return canMake;
  return canMake.filter(r =>
    r.name.toLowerCase().includes(term) ||
    r.ingredients.some(ing => ing.name.toLowerCase().includes(term))
  );
});
```

Kép 22 - makeableRecipes frontend metódus

getRecipes: A getRecipes() metódus a receptek és azok hozzávalóinak lekérését végzi a backendből. A kapott adatokat a frontend számára megfelelő formára alakítja, majd eltárolja az alkalmazás állapotában. Hiba esetén a felhasználó értesítést kap a betöltés sikertelenségéről.

```
getRecipes() {
  this.http.get<any[]>("http://127.0.0.1:8000/api/recipes/getAllRecipesWithIngredients")
    .pipe(
      map(response => response.map(recipe => this.mapRecipe(recipe))),
      catchError(err => {
        this.toast.show('Nem sikerült betölteni a recepteket.', 'error');
        return throwError(() => err);
      })
    ).subscribe(data => this._recipes.set(data));
}
```

Kép 23 - getRecipes frontend metódus

User-Storage Service

A UserStorageService osztály az Angular alkalmazásban a felhasználóhoz tartozó alapanyagok (raktárkészlet) kezeléséért felelős szolgáltatás. Feladata, hogy biztosítsa a kapcsolatot a frontend és a Laravel backend API között a felhasználó által birtokolt hozzávalók lekérése, hozzáadása és törlése során. A szolgáltatás Angular signal alapú állapotkezelést használ, így a felhasználói felületen automatikusan frissülnek az adatok.

addIngredientByName: Az addIngredientByName() metódus egy hozzávaló felhasználói készlethez történő hozzáadását végzi név alapján. Először lekéri a hozzávaló adatait, majd eltárolja azt a felhasználó raktárában a megadott mennyiséggel. Sikeres művelet után frissíti a hozzávalólistát, hiba esetén pedig értesítést jelenít meg.

```
addIngredientByName(userId: number, ingredientName: string, amount: number): Observable<any> {
  return this.getIngredientByName(ingredientName).pipe(
    switchMap((ingredient: Ingredient) =>
      this.storeUserIngredient({
        userId: userId,
        ingredientId: ingredient.id,
        amount: amount
      })
    ),
    tap(() => {
      this.toast.show(`${ingredientName} hozzáadva a raktárhoz!`, 'success');
      this.getUserIngredients();
    }),
    catchError(err => {
      this.toast.show('Nem sikerült hozzáadni az alapanyagot.', 'error');
      return throwError(() => err);
    })
  );
}
```

Kép 24 - addIngredientByName frontend metódus

getUserIngredients: A getUserIngredients() metódus a bejelentkezett felhasználó raktárkészletében található hozzávalókat kéri le a backendből. A kapott adatokat a frontend számára megfelelő formára alakítja, majd eltárolja az alkalmazás állapotában. Hiba esetén a rendszer naplózza a problémát.

```
getUserIngredients() {
  const userId = this.auth.loggedInUser()?.id;
  if (!userId) return;

  this.http.get<any[]>(`http://127.0.0.1:8000/api/users/${userId}/ingredients`)
    .pipe(
      map(response =>
        response.map(item => ({
          id: item.ingredientId,
          name: item.ingredient.name,
          amount: item.amount,
          unit: item.ingredient.unit
        }))
      ),
      catchError(err => {
        console.error('Raktár hiba:', err);
        return throwError(() => err);
      })
    )
    .subscribe(data => {
      this._userIngredients.set(data);
    });
}
```

Kép 25 - getUserIngredients frontend metódus

Backend

A ReceptPlus backendje Laravelben készült REST API, amely a felhasználókat, recepteket, hozzávalókat, kommenteket, valamint a felhasználó–hozzávaló és recept–hozzávaló kapcsolatokat kezeli. Az API JSON formátumban válaszol, és az üzleti logika a kontrollerekben található.

Modellek

A backendben található külön modell a felhasználókra (Users), kommentekre (Comment), hozzávalókra (Ingredients), receptekre (Recipe), valamint a sok-sok kapcsolatok megvalósítására Ingredient_Recipe és User_Ingredient modellek. Ezek a modellek controller fájlokkal lettek létrehozva.

Controllerek

Mindegyik controller fájlban található egy index (get), store (post), update (put) és destroy (delete) metódus. Ezek működésükben megegyeznek mindegyik modellben, valamint mind a pozitív és negatív ágak le lettek tesztelve a Thunder Client bővítménnyel.

Index metódus: Visszatér az adott modell összes adatával egy tömbben json fromátumban.

```
public function index()
{
    return users::All();
}
```

Kép 26 - Backend index metódus példa

Store metódus: Validálás után létrehoz új eleme(ke)t az adott táblában. Ha a validálás sikertelen, egy json hibaüzenet tér vissza. Ha függvény lefut sikeresen, egy sikert jelző üzenet tér vissza json formátumban.

```
public function store(Request $request)
{
    $validator = Validator::Make(
        $request->all(),
        [
            'passwrd' => 'required',
            'email' => 'required',
            'name' => 'required',
            'username' => 'required'
        ]
    );

    if ($validator->fails()) {
        return response()->json(["Hiba!" => "Legalább egy elemet kihagytál!"], 401);
    }

    $user = users::Create($request->all());
    return response()->json(["Sikeres feltöltés"], 201);

}
```

Kép 27 - Backend store metódus példa

Update metódus: Egy már meglévő elem frissítése az adatbázisban. Ha a validálás sikertelen, az elem nem frissül, és json hibaüzenet tér vissza. Ha sikeres a validálás, a mező frissül, és sikert jelző json üzenet tér vissza.

```
public function update(Request $request, users $userId)
{
    $validator = Validator::make($request->all(), [
        'passwrd' => 'required',
        'email' => 'required',
        'name' => 'required',
        'username' => 'required'
    ]);

    if ($validator->fails()) {
        return response()->json(
            ["Hiba!" => "Legalább egy elemet kihagytál!"],
            401
        );
    }

    $userId->update($request->all());

    return response()->json(
        ["Siker!" => "Felhasználó sikeresen frissítve!"],
        200
    );
}
```

Kép 28 - Backend update metódus példa

Destroy metódus: Egy meglévő elem törlése az adatbázisból. Ha az elem nem található, hibaüzenet tér vissza. Ha található, az elem törlődik, és egy sikert jelző json üzenet tér vissza.

```
public function destroy($userId)
{
    $user = users::find($userId);

    if (!$user) {
        return response()->json([
            "Hiba!" => "Nincs ilyen felhasználó az adatbázisban!"
        ], 404);
    }

    $user->delete();

    return response()->json([
        "Siker!" => "Ez a felhasználó sikeresen törölve!"
    ], 200);
}
```

Kép 29 - Backend destroy metódus példa

Az egyéb esetek lekezelésére speciális lekérdezések:

User Controller

GetById: Id alapján lekér egy felhasználót

```
public function getbyId($userId)
{
    $searched = Users::where('id', '=', $userId)->first();

    if (is_null($searched)) {
        return response()->json(
            ["Hiba!" => "Nincs ilyen felhasználó ezzel az Id-val!"],
            404
        );
    }

    return response()->json($searched, 200);
}
```

Kép 30 - User Controller GetById metódus

MoreThan_bodyweightKg: Lekéri azokat a felhasználókat, akiknek a testtömege nagyobb, mint paraméterben megadott szám

```
public function MoreThan_bodyweightKg($value)
{
    $searched = Users::where('bodyweightKg', '>', $value)->get();

    if ($searched->isEmpty()) {
        return response()->json(
            ["Hiba!" => "Nincs ennél az value-nál nagybobb bodyweightKg!"],
            404
        );
    }

    return response()->json($searched, 200);
}
```

Kép 31 - User Controller MoreThan_bodyweightKg metódus

LessThan_bodyweightKg: Lekéri azokat a felhasználókat, akiknek a testtömege kisebb, mint paraméterben megadott szám

```
public function LessThan_bodyweightKg($value)
{
    $searched = Users::where('bodyweightKg', '<', $value)->get();

    if ($searched->isEmpty()) {
        return response()->json(
            ["Hiba!" => "Nincs ennél az value-nál kisebb bodyweightKg!"],
            404
        );
    }

    return response()->json($searched, 200);
}
```

Kép 32 - User Controller LessThan_bodyweightKg metódus

login: A login() metódus a felhasználó bejelentkezését kezeli a backend oldalon. A megadott e-mail cím alapján megkeresi a felhasználót az adatbázisban, majd ellenőrzi a jelszót. Sikeres hitelesítés esetén visszaadja a felhasználó adatait, hiba esetén pedig megfelelő státuszkóddal és hibaüzenettel válaszol.

```
public function login(Request $request)
{
    $user = users::where('email', '=', $request->email)->first();

    if ($user && $user->passwrd == $request->passwrd) {
        return response()->json($user, 200);
    }

    return response()->json(['Hiba!' => 'Hibás email vagy jelszó!'], 401);
}
```

Kép 33 – User Controller login metódus

User_Ingredients Controller

getbyId: a felhasználó tárhelyéből id alapon egy hozzávaló lekérése

```
/**
 * Egy konkrét rekord lekérése + kapcsolatokkal
 */
public function getById($userId, $ingredientId)
{
    $userIngredient = User_Ingredient::with(['ingredient', 'user'])
        ->where('userId', $userId)
        ->where('ingredientId', $ingredientId)
        ->first();

    if (!$userIngredient) {
        return response()->json([
            "Hiba!" => "Nincs ilyen rekord az adatbázisban!"
        ], 404);
    }

    return response()->json($userIngredient, 200);
}
```

Kép 34 - User_Ingredients Controller getbyId metódus

Store: A store() metódus a felhasználó hozzávalóinak hozzáadását vagy frissítését kezeli. Először ellenőrzi az adatokat (userId, ingredientId, amount), majd megnézi, hogy a felhasználónak már van-e bejegyzése az adott hozzávalóból. Ha létezik, frissíti a mennyiséget, ha nem, új bejegyzést hoz létre. Minden esetben megfelelő státuszkóddal és üzenettel válaszol.

```php
public function store(Request $request)
{
    $validator = Validator::make($request->all(), [
        'userId' => 'required|exists:users,id',
        'ingredientId' => 'required|exists:ingredients,id',
        'amount' => 'required|numeric|min:1',
    ]);

    if ($validator->fails()) {
        return response()->json([
            'message' => 'Hibás adatokat küldtél.',
        ], 422);
    }

    $existing = DB::table('user_ingredient')
        ->where('userId', $request->userId)
        ->where('ingredientId', $request->ingredientId)
        ->first();

    if ($existing) {
        DB::table('user_ingredient')
            ->where('userId', $request->userId)
            ->where('ingredientId', $request->ingredientId)
            ->update([
                'amount' => $existing->amount + $request->amount,
            ]);

        return response()->json([
            'message' => 'A hozzávaló mennyisége sikeresen frissítve.',
        ], 200);
    }

    DB::table('user_ingredient')->insert([
        'userId' => $request->userId,
        'ingredientId' => $request->ingredientId,
        'amount' => $request->amount,
    ]);

    return response()->json([
        'message' => 'A hozzávaló sikeresen hozzáadva.',
    ], 201);
}
```

Kép 35 - User_Ingredients Controller store metódus

Recipe Controller

getbyId: egy recept lekérése Id alapon

```
public function getbyId($recipeId)
{
    $searched = Recipe::where('id', '=', $recipeId)->first();

    if (is_null($searched)) {
        return response()->json(
            ["Hiba!" => "Nincs ilyen recept ezzel az Id-val!"
            404
        );
    }

    return response()->json($searched, 200);
}
```

Kép 36 - Recipe Controller getbyId metódus

makeRecipe: A makeRecipe() metódus a recept „elkészítését” kezeli a felhasználó szempontjából. Ellenőrzi a felhasználó és a recept létezését, majd tranzakcióban levonja a recepthez szükséges hozzávalókat a felhasználó készletéből. Ha egy hozzávaló elfogy, törli, ha marad belőle, frissíti a mennyiséget. Ezután kiszámolja a recept kalóriatartalmát, és hozzáadja a felhasználóhoz. Siker esetén visszaadja a frissített felhasználói adatokat és a megmaradt hozzávalókat, hiba esetén pedig visszagörgeti a műveletet és hibát küld vissza.

```php
public function makeRecipe($userId, $recipeId)
{
    $user = users::find($userId);
    $recipe = Recipe::with('ingredients')->find($recipeId);

    if (! $user) {
        return response()->json([
            'message' => 'A funkció eléréséhez jelentkezz be!',
        ], 404);
    }

    if (! $recipe) {
        return response()->json([
            'message' => 'Recept nem található!',
        ], 404);
    }

    DB::beginTransaction();

    try {

        foreach ($recipe->ingredients as $recipeIngredient) {
            $userIngredient = DB::table('user_ingredient')
                ->where('userId', $user->id)
                ->where('ingredientId', $recipeIngredient->id)
                ->first();

            if (! $userIngredient) {
                continue;
            }

            $remainingAmount = $userIngredient->amount - $recipeIngredient->pivot->amount;

            if ($remainingAmount <= 0) {
                DB::table('user_ingredient')
                    ->where('userId', $user->id)
                    ->where('ingredientId', $recipeIngredient->id)
                    ->delete();
            } else {
                DB::table('user_ingredient')
                    ->where('userId', $user->id)
                    ->where('ingredientId', $recipeIngredient->id)
                    ->update([
                        'amount' => $remainingAmount,
                    ]);
            }
        }

        $recipeCalories = $this->calculateRecipeCalories($recipe);
        $user->caloriesEaten = ($user->caloriesEaten ?? 0) + $recipeCalories;
        $user->save();

        DB::commit();

        $remainingIngredients = DB::table('user_ingredient')
            ->join('ingredients', 'ingredients.id', '=', 'user_ingredient.ingredientId')
            ->where('user_ingredient.userId', $user->id)
            ->select(
                'ingredients.id',
                'ingredients.name',
                'ingredients.unit',
                'user_ingredient.amount'
            )
            ->get();

        return response()->json([
            'user' => $user,
            'remainingIngredients' => $remainingIngredients,
        ], 200);

    } catch (\Exception $e) {
        DB::rollBack();

        return response()->json([
            'message' => 'Hiba történt a recept elkészítése közben',
            'error' => $e->getMessage(),
        ], 500);
    }
}
```

Kép 37 - Recipe Controller makeRecipe metódus

Comments Controller

getbyId: recept lekérése Id alapon

```php
/**
 * Komment lekérése ID alapján
 */
public function getById($commentId)
{
    $searched = Comment::where('id', '=', $commentId)->first();

    if (is_null($searched)) {
        return response()->json(
            ["Hiba!" => "Nincs ilyen komment ezzel az Id-val!"],
            404
        );
    }

    return response()->json($searched, 200);
}
```

Kép 38 - Comments Controller getbyId metódus

getByRecipeId: kommentek lekérése recept Id alapján

```php
/**
 * Egy recepthez tartozó kommentek
 */
public function getByRecipeId($recipeId)
{
    $comments = Comment::where('recipeId', '=', $recipeId)->get();

    if ($comments->isEmpty()) {
        return response()->json(
            ["Hiba!" => "Ehhez a recepthez nincs komment!"],
            404
        );
    }

    return response()->json($comments, 200);
}
```

Kép 39 - Comments Controller getByRecipeId metódus

getByUserId: kommentek lekérése felhasználó Id alapján

```
/**
 * Egy user kommentjei
 */
public function getByUserId($userId)
{
    $comments = Comment::where('userId', '=', $userId)->get();

    if ($comments->isEmpty()) {
        return response()->json(
            ["Hiba!" => "Ez a felhasználó még nem kommentelt!"],
            404
        );
    }

    return response()->json($comments, 200);
}
```

Kép 40 - Comments Controller getByUserId metódus

Ingredients Controller

getById: Hozzávaló lekérése Id alapján

```
/**
 * Ingredient lekérése ID alapján
 */
public function getById($ingredientId)
{
    $searched = Ingredients::where('id', '=', $ingredientId)->first();

    if (is_null($searched)) {
        return response()->json(
            ["Hiba!" => "Nincs ilyen alapanyag ezzel az Id-val!"],
            404
        );
    }

    return response()->json($searched, 200);
}
```

Kép 41 - Ingredients Controller getById metódus

getFruits: csak gyümölcsök lekérdezése

```
/**
 * Csak gyümölcsök
 */
public function getFruits()
{
    $ingredients = Ingredients::where('isFruit', '=', true)->get();

    if ($ingredients->isEmpty()) {
        return response()->json(
            ["Hiba!" => "Nincs egyetlen gyümölcs sem!"],
            404
        );
    }

    return response()->json($ingredients, 200);
}
```

Kép 42 - Ingredients Controller getFruits metódus

getNonFruits: alapanyagok lekérése gyümölcsöket kizárva

```
/**
 * Nem gyümölcs alapanyagok
 */
public function getNonFruits()
{
    $ingredients = Ingredients::where('isFruit', '=', false)->get();

    if ($ingredients->isEmpty()) {
        return response()->json(
            ["Hiba!" => "Nincs egyetlen nem gyümölcs alapanyag sem!"],
            404
        );
    }

    return response()->json($ingredients, 200);
}
```

Kép 43 - Ingredients Controller getNonFruits metódus

MoreThan_kcalPerGram: lekéri azokat a hozzávalókat, amik több kalóriával rendelkeznek gramonként mint a megadott érték

```
/**
 * Kcal/gram nagyobb mint
 */
public function MoreThan_kcalPerGram($value)
{
    $ingredients = Ingredients::where('kcalPerGram', '>', $value)->get();

    if ($ingredients->isEmpty()) {
        return response()->json(
            ["Hiba!" => "Nincs ennél nagyobb kcal/gram érték!"],
            404
        );
    }

    return response()->json($ingredients, 200);
}
```

Kép 44 - Ingredients Controller MoreThan_kcalPerGram metódus

LessThan_kcalPerGram: lekéri azokat a hozzávalókat, amik kevesebb kalóriával rendelkeznek gramonként mint a paraméterben megadott érték

```
/**
 * Kcal/gram kisebb mint
 */
public function LessThan_kcalPerGram($value)
{
    $ingredients = Ingredients::where('kcalPerGram', '<', $value)->get();

    if ($ingredients->isEmpty()) {
        return response()->json(
            ["Hiba!" => "Nincs ennél kisebb kcal/gram érték!"],
            404
        );
    }

    return response()->json($ingredients, 200);
}
```

Kép 45 - Ingredients Controller LessThan_kcalPerGram metódus

Ingredients_Recipe Controller

getByRecipe: Egy adott recepthez tartozó hozzávalók lekérése. A lekérés a recept azonosítója alapján történik.

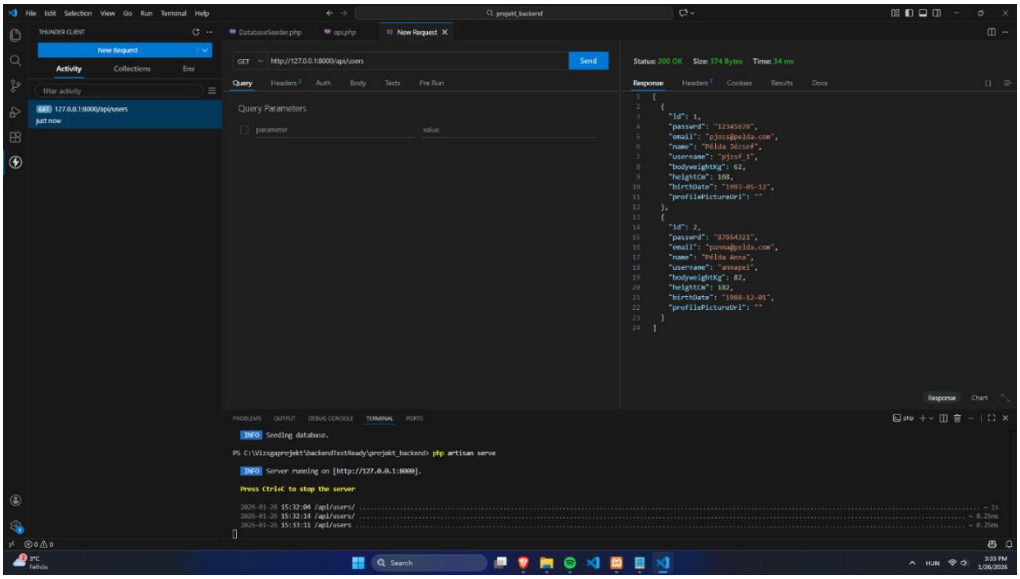
```
/**
 * Lekérés recept alapján
 */
public function getByRecipe($recipeId)
{
    $ingredients = DB::table('ingredient_recipe as ir')
        ->join('ingredients as i', 'ir.ingredientId', '=', 'i.id')
        ->where('ir.recipeId', $recipeId)
        ->select(
            'i.id as ingredientId',
            'i.name as ingredientName',
            'ir.gramAmount'
        )
        ->get();

    if ($ingredients->isEmpty()) {
        return response()->json(
            ["Hiba!" => "Ehhez a recepthez nincs hozzávaló!"],
            404
        );
    }

    return response()->json($ingredients, 200);
}
```

Kép 46 - Ingredients_Recipe Controller getByRecipe metódus

Tesztelés

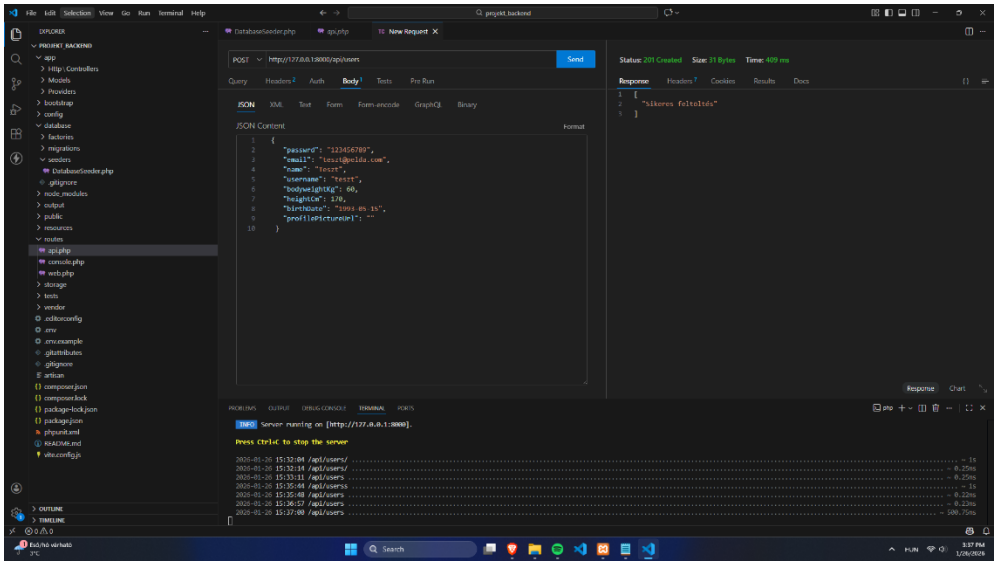
Backend tesztek

Index metódus: Pozitív esetben 200-as státuszkóddal térnek vissza az index metódusok, negatív eset nem lehetséges a lekérés egyszerűsége miatt, hibás meghívásra nincs lehetőség.

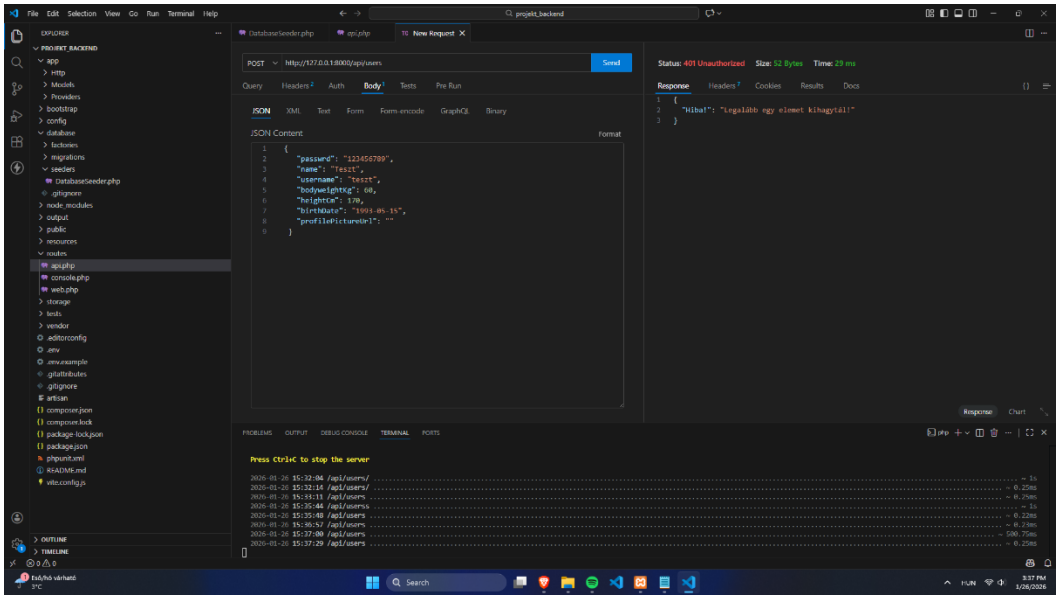


Kép 47 - Backend Index metódus teszt példa (csak pozitív eset lehetséges)

Store metódus: A store metódus sikeresen lefutás esetén 201-es státuszkóddal tér vissza, ha a frontend az összes kötelező adatot elküldte. Hibás lefutás esetén 401-es státuszkóddal tér vissza, kihagyott elemek esetén.

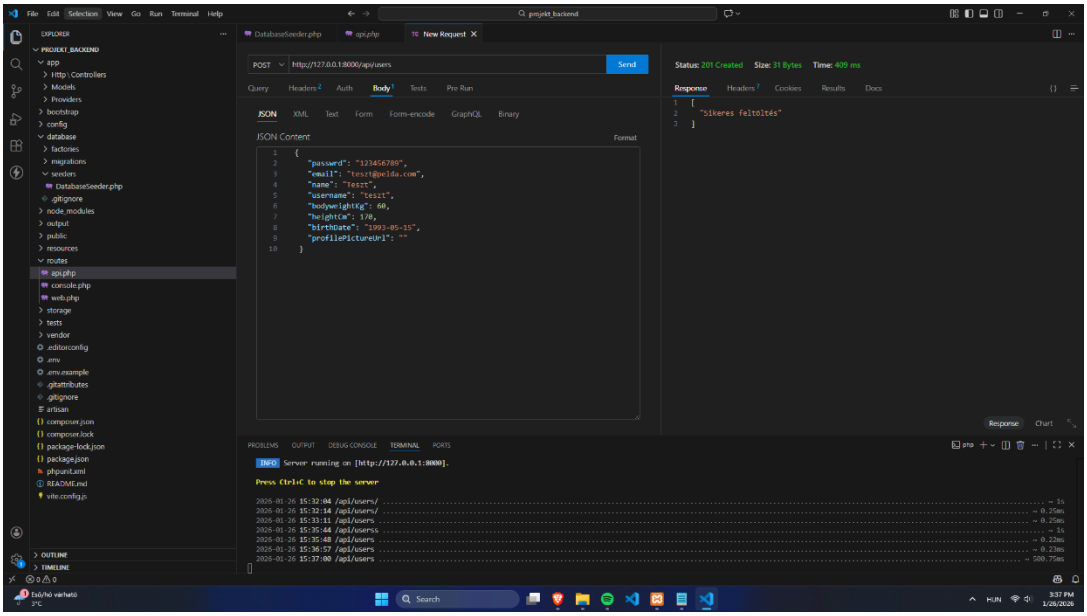


Kép 48 - Backend Store metódus teszt példa - pozitív

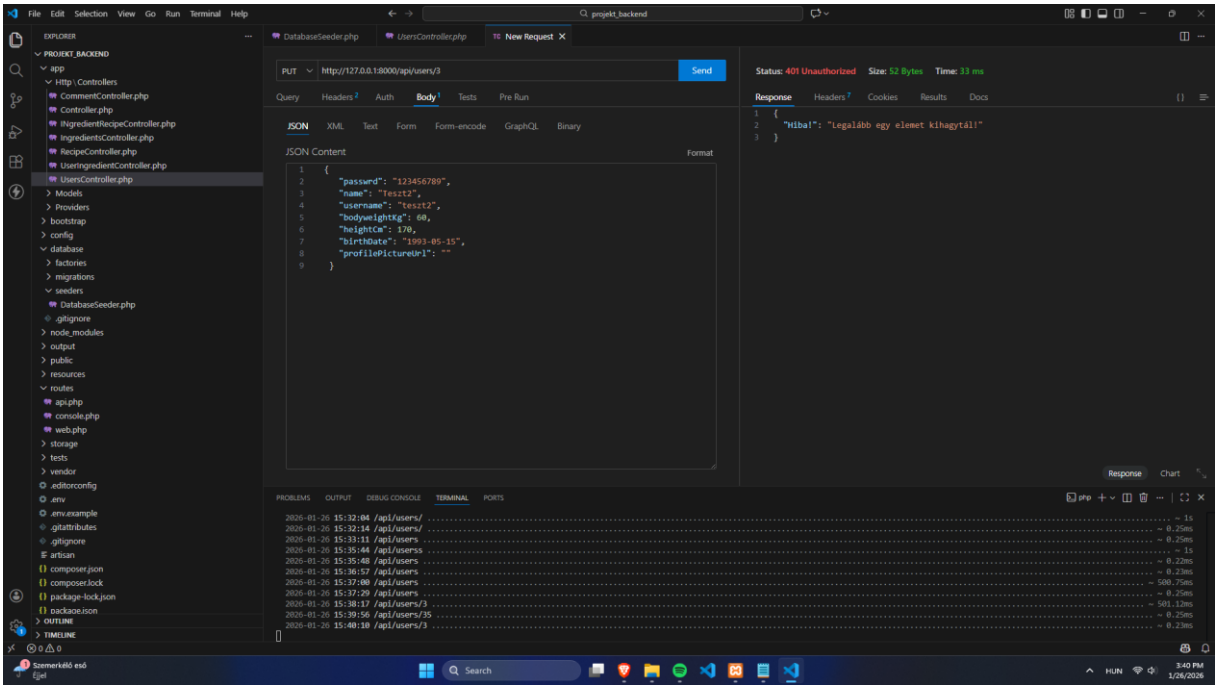


Kép 49 - Backend Store metódus teszt példa – negatív

Update metódus: A store metódus tesztjéhez hasonló, viszont egy meglévő elem adatait írja át, validálással. A státuszkódok megegyeznek a store metódusával.

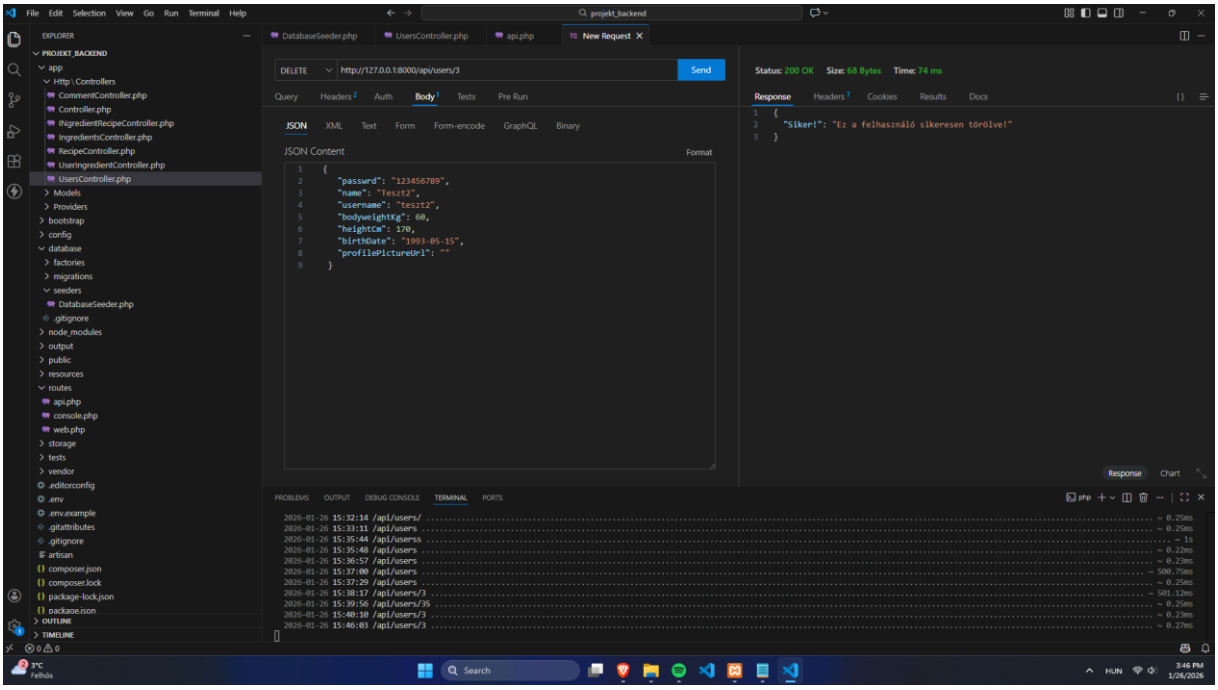


Kép 50 - Backend Update metódus teszt példa - pozitív

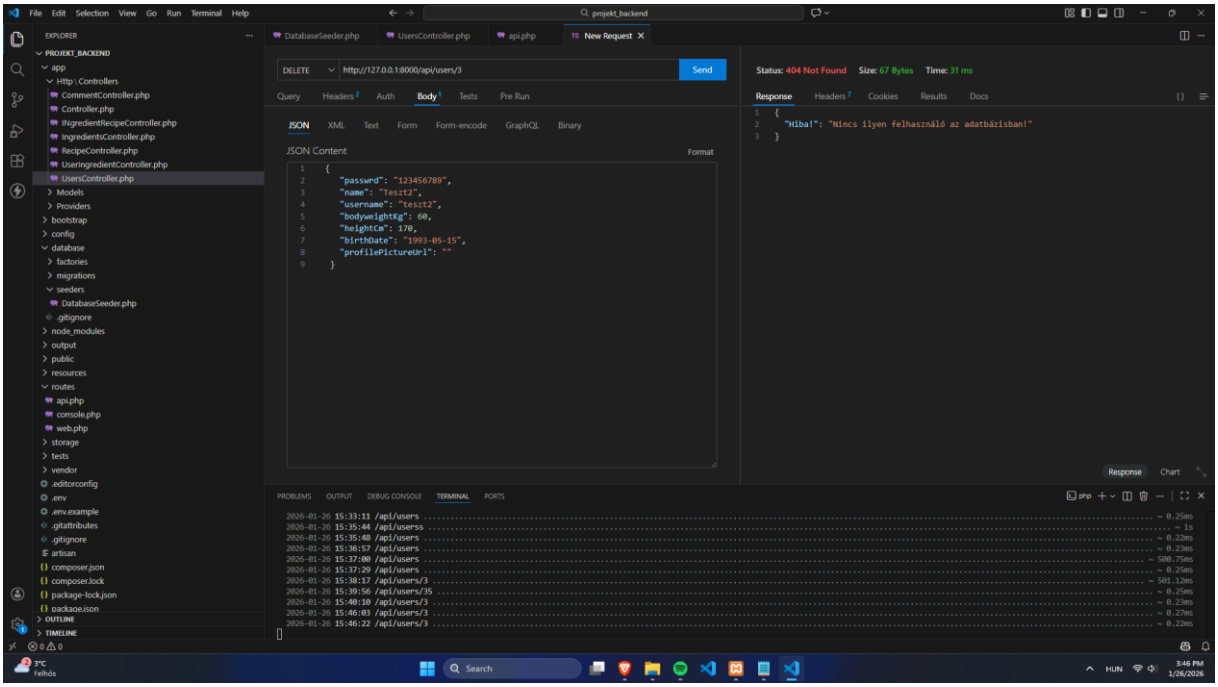


Kép 51 - Backend Update metódus teszt példa - negatív

Destroy metódus: A destroy metódus sikeres lefutás esetén 200-as státuszkóddal tér vissza. Nem talált mező esetén 404-es kóddal a teszt elbukik.



Kép 52 - Backend Destroy metódus teszt példa - pozitív



Kép 53 - Backend Destroy metódus teszt példa - negatív

Frontend tesztek

Az angular ng test parancs eredménye:

```
Initial chunk files                                          | Names                                                    |  Raw size
styles.css                                                   | styles                                                   | 380.81 kB |
scripts.js                                                   | scripts                                                  | 107.73 kB |
spec-app-add-recipe-add-recipe.js                            | spec-app-add-recipe-add-recipe                           |  36.81 kB |
spec-app-userstorage-userstorage.js                          | spec-app-userstorage-userstorage                         |  35.64 kB |
spec-app-accountinfo-accountinfo.js                          | spec-app-accountinfo-accountinfo                         |  32.79 kB |
spec-app-login-login.js                                      | spec-app-login-login                                     |  32.77 kB |
spec-app-userrecepies-userrecepies.js                        | spec-app-userrecepies-userrecepies                       |  30.66 kB |
spec-app-recipe-full-recipe-full.js                          | spec-app-recipe-full-recipe-full                         |  27.87 kB |
chunk-LPVJKUCB.js                                            | -                                                        |  27.36 kB |
spec-app-aboutuscomponent-aboutuscomponent.js                | spec-app-aboutuscomponent-aboutuscomponent               |  23.45 kB |
spec-app-recepiesfrominventory-recepiesfrominventory.js      | spec-app-recepiesfrominventory-recepiesfrominventory     |  23.44 kB |
spec-app-all-recipes-all-recipes.js                          | spec-app-all-recipes-all-recipes                         |  22.70 kB |
spec-app-register-register.js                                | spec-app-register-register                               |  20.42 kB |
spec-app-resetpasswordcomponent-resetpasswordcomponent.js    | spec-app-resetpasswordcomponent-resetpasswordcomponent   |  16.94 kB |
chunk-G3ATBCZM.js                                            | -                                                        |  16.66 kB |
spec-app-app.js                                              | spec-app-app                                             |   9.90 kB |
chunk-TZGVPLVK.js                                            | -                                                        |   5.51 kB |
chunk-TGQLNVO2.js                                            | -                                                        |   5.12 kB |
chunk-JAHGJQV6.js                                            | -                                                        |   4.92 kB |
chunk-RSTWSNOR.js                                            | -                                                        |   4.75 kB |
chunk-YF4DV2GR.js                                            | -                                                        |   3.27 kB |
chunk-PZPMZHVB.js                                            | -                                                        |   3.13 kB |
chunk-JT52QV3B.js                                            | -                                                        |   2.01 kB |
chunk-2272NKRY.js                                            | -                                                        |   1.76 kB |
spec-app-authguard-service.js                                | spec-app-authguard-service                               |   1.67 kB |
init-testbed.js                                              | init-testbed                                             |   1.21 kB |
spec-app-searchbar-searchbar.js                              | spec-app-searchbar-searchbar                             | 741 bytes |
spec-app-toastwindowcomponent-toastwindowcomponent.js        | spec-app-toastwindowcomponent-toastwindowcomponent       | 739 bytes |
spec-app-navbar-navbar.js                                    | spec-app-navbar-navbar                                   | 657 bytes |
spec-app-footer-footer.js                                    | spec-app-footer-footer                                   | 597 bytes |
spec-app-recipe.service.js                                   | spec-app-recipe.service                                  | 550 bytes |
spec-app-user-storage-service.js                             | spec-app-user-storage-service                            | 547 bytes |
spec-app-comment-service.js                                  | spec-app-comment-service                                 | 495 bytes |
spec-app-auth-service.js                                     | spec-app-auth-service                                    | 480 bytes |
spec-app-toastwindowservice.js                               | spec-app-toastwindowservice                              | 471 bytes |

                                                             | Initial total                                            | 884.59 kB
```

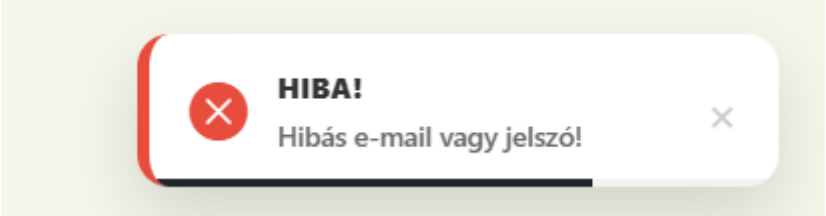
Kép 54 - Frontend ng test parancs (1. rész)

```
 Test Files  22 passed (22)
      Tests  22 passed (22)
   Start at  16:17:03
   Duration  1.12s (transform 62ms, setup 270ms, import 70ms, tests 79ms, environment 551ms)

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

Kép 55 - Frontend ng test parancs (2. rész)

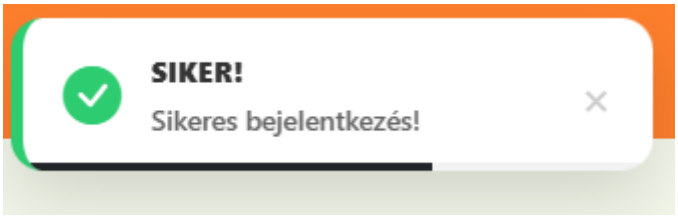
A felületben megjelenő hibaüzenetek felugró kártyákon jelennek meg. Ugyanez használt a sikeres események feltüntetésére.

Hibás bejelentkezés esetén dobott hibakód:



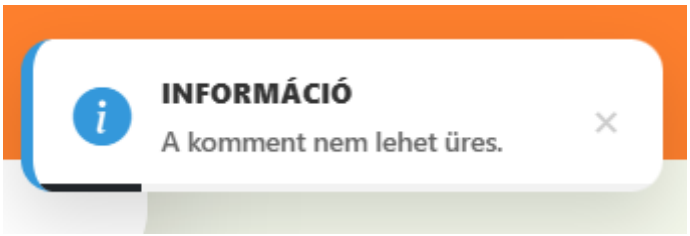
Kép 56 - HIbás email/jelszó üzenet

Sikeres bejelentkezés esete:



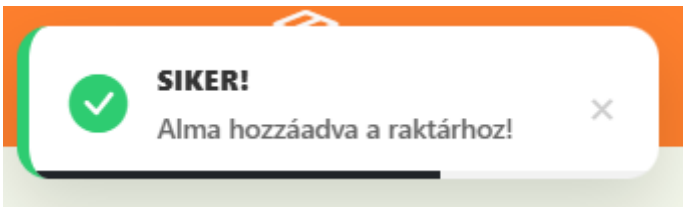
Kép 57 - Bejelentkezéskori üzenet

Üres komment írása:



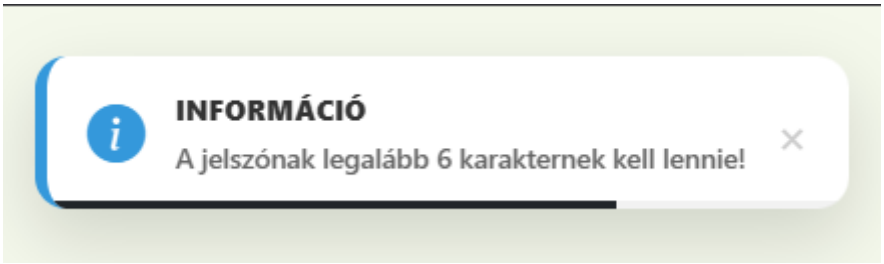
Kép 58 - Komment létrehozás információ

Sikeres raktárhoz adás:



Kép 59 - Raktárhoz adás üzenet

Hibás jelszó:



Kép 60 - Hibás jelszó létrehozás üzenet

Továbbfejlesztési lehetőségek

Az alkalmazás továbbfejlesztésére több lehetőség is elérhető. Az adatbázist és a backendet a webalkalmazás össze kapcsolhatná otthoni okos eszközökkel (pl.: okos hűtő). A kommenteket a felhasználók neveivel meg lehetne jeleníteni, illetve implementálni lehetne a meglévő receptek és felhasználói alapanyagok (tároló) felhasználói felületen való módosítását. Az alapanyagok listáját felhasználók által is bővíthetővé lehetne tenni, az ez, és a receptek hozzáadásánál pedig bizonyos szavak használatát meg lehetne tiltani.

Irodalomjegyzék

Mesterséges Intelligencia segítség: https://chatgpt.com/

Bootstrap dokumentáció: https://getbootstrap.com/docs/5.3/customize/overview/

Angular Material dokumentáció: https://material.angular.dev/components/categories

Laravel dokumentáció: https://laravel.com/docs/13.x